Fpt University

TRƯỜNG ĐẠI HỌC FPT

CHAY XANH

Software Requirement Document

Vegetarian Nutrition & Healthy Living Platform

– HCM, Nov 2026 –

RECORD OF CHANGES

Date	A* M, D	In charge	Change Description
07/Oct	A	Minh Hậu, Trường Phú, Minh Hoàng	Initial project setup: React client (home, navbar, recipes, community pages, chat UI), Express server and FastAPI AI service with the first chatbot.
08/Oct	A	Minh Hậu	Community API for blogs, videos, comments and votes; keyword search and recommendations.
08–10/Oct	A	Minh Hoàng	AI nutrition chatbot with SSE streaming; weekly meal planner (BMR/TDEE targets, diet and allergen validation, pantry); shared guest daily quota and rate limiting; LLM unified on DeepSeek; restaurant search API using ZIOMAP.
11/Oct	A	Minh Hậu	PostgreSQL database with Prisma, JWT authentication and role authorization (ADMIN/USER), Zod validators; seed data replaces mock data.
12/Oct	M	Minh Hậu, Minh Hoàng	Schema extended with ingredient details, restaurant reviews, menu prices and ingredient images stored in PostgreSQL; client connected to AI chat and meal plan dialog; Leaflet restaurant map.
14–22/Oct	A	Minh Hậu	Administrator portal: dashboard, members, posts, comments, categories; soft delete, restore and data purge; database ERD.
22/Oct	M	Minh Hoàng	AI calls moved behind the Express API: member AI history (ai_generations), AI logs, internal API key between services, guest quota administration.
23–26/Oct	A	Minh Hậu	Change password, weight tracker and health history, notification system; restaurant data model extended.
26/Oct	A	Minh Hoàng	YouTube video recipe summary (jobs, background worker, progress in chat); signed guest session cookie for AI quota.
26–29/Oct	A	Minh Hoàng	Food scanner: analyse dish photos and ingredient labels against the selected diet.
28–29/Oct	A	Minh Hoàng	Automated AI content moderation (jobs, worker, admin review); apply AI weekly plan to the journal; AI-generated ingredient dictionary entries.
3–7/Nov	M	Trường Phú	Integration audit and repairs (admin members/reports, session refresh, profile and journal sync, pagination); password recovery by e-mail OTP via SMTP.
04/Nov	M	Minh Hoàng	Video recipe summary changed from audio transcription to Vietnamese YouTube captions
5–7/Nov	A	Minh Hậu	Detailed ingredient nutrition, ml-to-gram conversion, dish calorie totals; fiber and automatic macros in the journal.
07/Nov	A	Minh Hoàng	Moderation of uploaded post images; AI per-100 g nutrition for ingredient lookup.
07/Nov	D	Minh Hậu	Removed legal document management; added PENDING_MODERATION post status and new admin layout (header, sidebar).
08/Nov	A	Minh Hậu	Restaurant proposal form (GPS, opening hours, price range, separate kitchen) and vegetarian place filtering.

*A - Added M - Modified D - Deleted

Table of Contents

I. Overview..... 5
1. Introduction..... 5
2. Existing Systems..... 6
2.1 HappyCow..... 6
2.2 MyFitnessPal.....6
2.3 Cookpad.....7
3. Business Process.....7
3.1 AI meal planning and journal application.....7
3.2 Community publication and automated moderation.....8
3.3 Violation report handling..... 9
4. System Functions.....9
5. Entity Relationship Diagram..... 12
6. State transition diagram.....13
II. Functional Requirements..... 15
1. Account and Access.....15
a. Register.....15
b. Login and logout.....15
c. Forgot and reset password..... 16
2. AI Nutrition Assistant..... 17
a. Nutrition chat..... 17
b. Generate weekly meal plan.....18
c. Guest AI quota..... 19
d. YouTube video recipe summary.....20
3. Food Knowledge.....20
a. Recipe catalog.....20
b. Ingredient dictionary and AI lookup.....21
c. Food swap.....21
d. Food scanner..... 22
4. Restaurant Discovery.....23
a. Search restaurants on the map.....23
b. Review a restaurant.....24
c. Suggest a new place.....24
5. Community.....25
a. Publish blog and video posts..... 25
b. Comment and vote.....25

c. Report a violation..... 26
d. Automated content moderation..... 26
6. Personal Health..... 26
a. Profile and health profile..... 26
b. Food journal..... 27
c. Weekly plan and applying AI plans..... 28
d. Notifications..... 29
7. Administration..... 29
a. Dashboard and system health..... 29
b. Member management..... 30
c. Post and comment management..... 31
d. Category management..... 32
e. AI moderation queue..... 33
f. Violation report handling..... 34
g. AI monitor and AI management..... 35
III. Database Design..... 36
1. Database Schema..... 36
2. Table Detail..... 40
2.1 Identity, security and AI governance..... 40
2.2 Health profile and preferences..... 43
2.3 Ingredients and knowledge..... 44
2.4 Recipes, meal planning and diary..... 46
2.5 Restaurant discovery..... 48
2.6 Community and background processing..... 49
2.7 Enumerations..... 53

I. Overview

1. Introduction

Chay Xanh is a web platform that supports people who follow or want to try a vegetarian diet. It brings together vegetarian recipe discovery, an ingredient dictionary with nutrition facts, ingredient substitutions, food and label checking with AI, a map of vegetarian restaurants and ingredient shops, an AI nutrition assistant that can generate weekly menus, a personal food journal and health profile, and a moderated community for blogs and cooking videos.

The problem it addresses: vegetarians in Vietnam have to combine several unrelated tools (recipe sites, calorie counters, maps, social groups). Those tools rarely understand Vietnamese vegetarian rules such as Buddhist vegetarianism without the five pungent vegetables, cannot check a dish against the user's own diet and allergies, and give no trustworthy place to share content.

Users of the system

- **Guest:** an unauthenticated visitor. Browses public content (recipes, ingredients, swaps, restaurants, community feed) and may try the AI features within a shared daily quota (default 3 AI turns per day).
- **Member (USER, role = 2):** a registered account. Manages a health profile and allergies, keeps a food journal and weekly plan, applies AI meal plans, publishes posts, comments, votes, reviews restaurants and reports violations.
- **Administrator (ADMIN, role = 1):** operates the platform: dashboard, members, posts, comments, categories, AI moderation queue, AI monitoring and management, and violation reports.

Operating environment

Service	Technology	Port
client/	React 19, Vite 8, React-Bootstrap 2, Bootstrap 5, React Router, Leaflet (map)	5173
server/	Node.js >= 20, Express 5, Prisma 6 ORM, Zod validation, JWT authentication, Pino logging	5000
ai_service/	Python >= 3.11, FastAPI, Pydantic v2, LangChain, DeepSeek LLM client	8000
Database	PostgreSQL >= 15 (41 tables managed by Prisma Migrate)	5432

Constraints, assumptions and dependencies

- **Local deployment:** all services run directly on the developer's machine; no container or cloud deployment is in scope.
- **Service boundary:** only the Express API and its workers access PostgreSQL. The AI service never connects to the database and accepts calls only with the internal header X-Internal-Api-Key.
- **Unified API contract:** REST responses use {success, message, data, meta}; errors use {success: false, message, errorCode, errors[]}. Date-times are exchanged as ISO 8601 UTC strings.
- **Real data only:** screens show persisted records or live provider results; when a provider fails the UI shows an error, never invented content.
- **Advisory nutrition:** calorie, macro and diet-compliance results are AI or database estimates and do not replace professional medical or nutrition advice.
- **External dependencies:** DeepSeek LLM API, ZIOMAP place search, YouTube (metadata and captions), an SMTP server for OTP e-mails, OSM France / CARTO map tiles and Google Maps direction links. Features that use them require valid configuration and network access.

System context

The context diagram below shows the boundary of Chay Xanh, the three human actors and the external systems it exchanges data with.

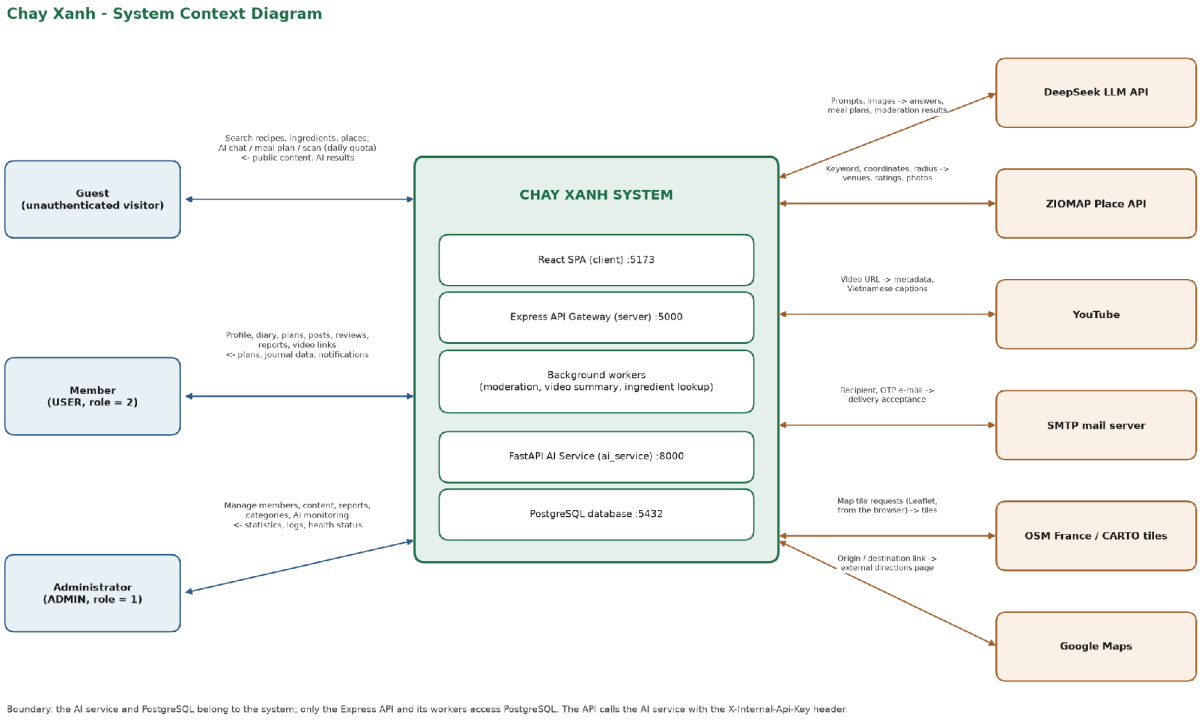


Figure 1. Chay Xanh system context diagram

2. Existing Systems

2.1 HappyCow

- **Link:** https://www.happycow.net
- **Description:** a well-known directory of vegan and vegetarian restaurants and shops with web and mobile apps.
- **Actors:** diners and travellers, community reviewers, venue owners.
- **Features:** location-based venue search, vegan / vegetarian / veg-options classification, user reviews and photos, city guides.
- **Pros:** large crowdsourced venue database and active community; clear venue classification.
- **Cons:** no personal nutrition tracking, no AI checking of dishes against a diet, little coverage of Vietnamese Buddhist vegetarian rules; mainly English content.
- **What we learn:** venue classification, map + list search, separate-kitchen information and reviews (used in our Restaurants feature).

2.2 MyFitnessPal

- **Link:** https://www.myfitnesspal.com
- **Description:** a popular calorie counter and food diary application.
- **Actors:** people tracking weight or fitness goals, coaches.
- **Features:** food diary by meal, calorie and macronutrient goals, large food database, weight tracking; meal planning in paid plans.
- **Pros:** fast logging and clear daily goal versus intake summaries.

- **Cons:** built for general diets; does not understand vegetarian sub-diets, allergies to hidden animal ingredients or the five pungent vegetables; key features behind a subscription; no restaurant discovery or vegetarian community.
- **What we learn:** four meal groups per day, daily totals against personal targets and weekly calorie history (used in our Journal).

2.3 Cookpad

- **Link:** https://cookpad.com/vn
- **Description:** a community recipe-sharing platform that is widely used in Vietnam.
- **Actors:** home cooks who publish recipes, readers, community moderators.
- **Features:** publish recipes with photos, search by ingredient, comments and reactions, saved collections.
- **Pros:** large Vietnamese recipe community and simple publishing flow.
- **Cons:** not specialised for vegetarians, no nutrition calculation per diet and no automated checking of posted content.
- **What we learn:** community feed, publishing flow and reactions (used in our Community feature, extended with AI moderation).

3. Business Process

The three main business processes are shown as swimlane diagrams. Each lane is a role or a system component that executes the steps.

3.1 AI meal planning and journal application

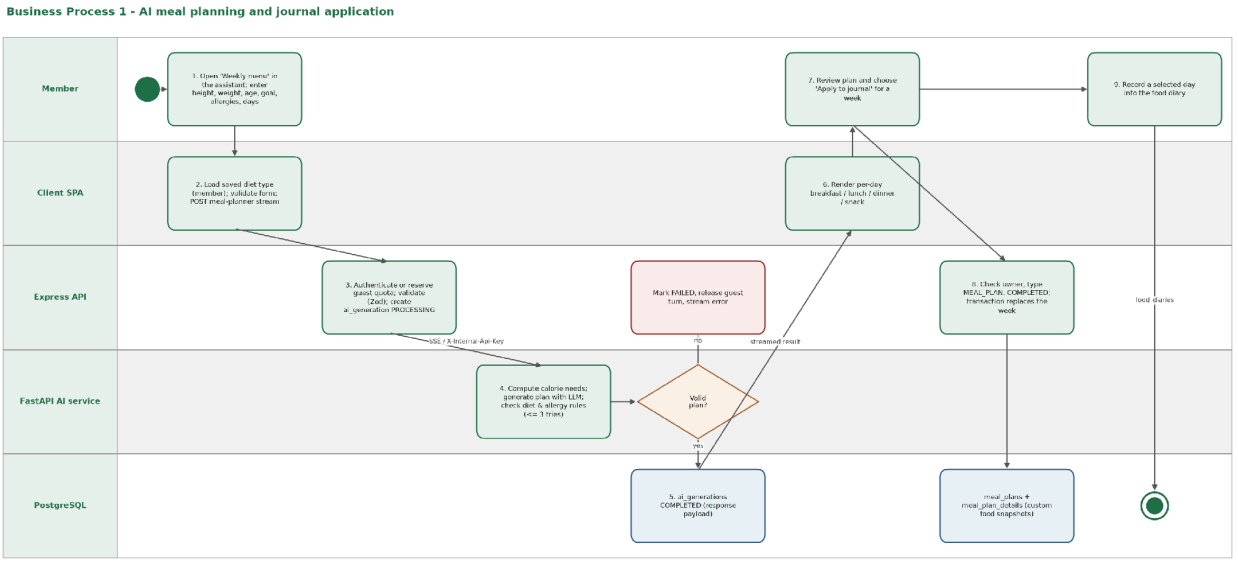


Figure 2. Swimlane - AI meal planning and journal application

#	Process Step	Description
1	Enter planning data	The user opens 'Weekly menu' in the AI assistant and enters height, weight, age, gender, goal, activity level, allergies, optional pantry ingredients and 1-7 days.
2	Prepare request	For a member the client reloads the diet type saved in the health profile; a guest uses the diet chosen on the home page. The form is validated and sent as a streaming request.
3	Authorise and record	The API validates the payload, authenticates the member or reserves one guest AI turn, and creates an ai_generations record with status PROCESSING.
4	Generate plan	The AI service computes the energy target and asks the LLM for breakfast, lunch, dinner and optional snack per day; diet, allergy and day-count rules are checked, with up to three attempts.

#	Process Step	Description
5	Save result	A valid plan is saved as COMPLETED and streamed to the client. On failure the generation is FAILED, the guest turn is released and an error is shown.
6	Review plan	The client renders the plan by day. Generating a plan does not change the journal.
7	Apply to journal	A member chooses 'Apply to journal' and a week. The API checks ownership, type and status, then replaces that Monday-Sunday week in meal_plans / meal_plan_details in one transaction.
8	Record a day	The member may record a planned day into the food diary as a separate explicit action.

3.2 Community publication and automated moderation

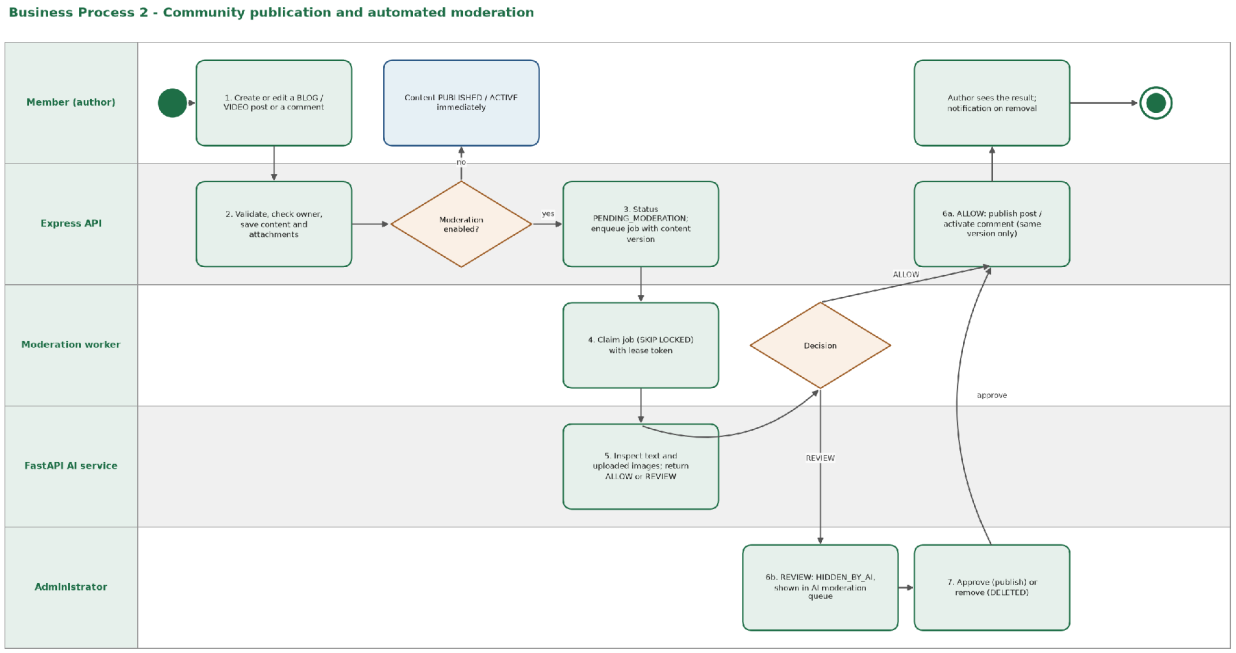


Figure 3. Swimlane - community publication and automated moderation

#	Process Step	Description
1	Create or edit content	A member creates or edits a BLOG or VIDEO post (title, content, categories, up to 6 attachments) or writes a comment.
2	Validate and save	The API validates the input, checks ownership and stores the post, its subtype and attachments.
3	Queue moderation	When CONTENT_MODERATION_ENABLED is true the content becomes PENDING_MODERATION and a content_moderation_jobs record is created with the current content version; otherwise it is published immediately.
4	Process job	A background worker claims the job (FOR UPDATE SKIP LOCKED) with a lease token, sends text and uploaded images to the AI service and receives ALLOW or REVIEW.
5	Apply decision	ALLOW publishes the post or activates the comment. REVIEW sets HIDDEN_BY_AI. The result is applied only if the content version still matches, so an old result cannot overwrite a newer edit.
6	Administrator review	Hidden or failed items appear in the AI moderation tab; the administrator approves (publish) or removes (DELETED) them.

3.3 Violation report handling

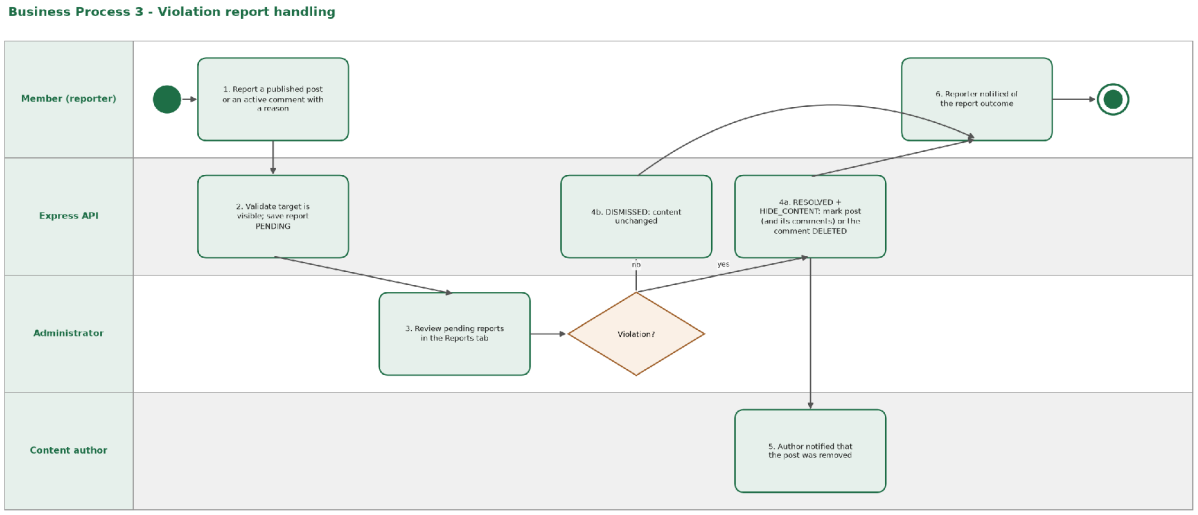


Figure 4. Swimlane - violation report handling

#	Process Step	Description
1	Submit report	A member reports a published post or an active comment and gives a reason.
2	Save report	The API checks that the target is still visible and stores the report with status PENDING.
3	Review	The administrator opens the Reports tab, reads the reason and the reported content.
4	Resolve or dismiss	RESOLVED with HIDE_CONTENT deletes the comment, or the post together with its comments; DISMISSED keeps the content unchanged.
5	Notify	The post author is notified when the post is removed; the reporter is notified of the outcome.

4. System Functions

a. Screen Flow

The SPA uses Vietnamese URL paths. Public screens are reachable from the header menu; member screens require login and the administrator portal requires the ADMIN role. The AI assistant is a floating widget on every site page.

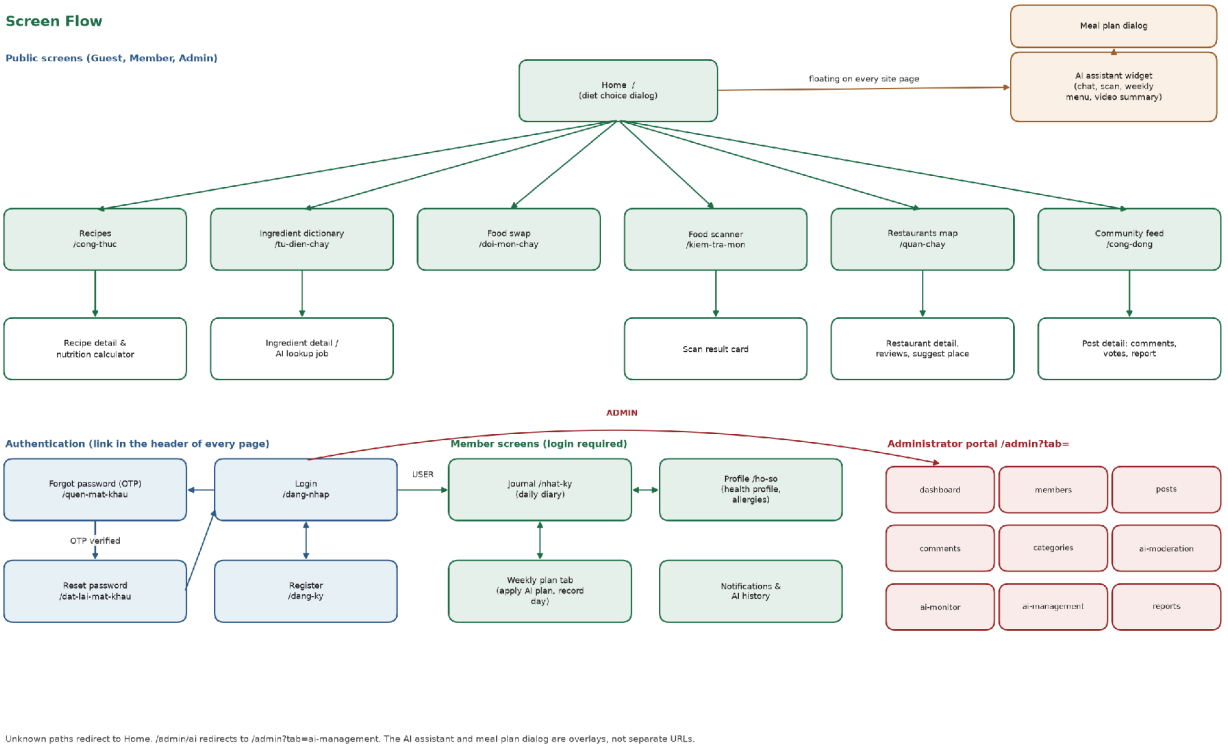


Figure 5. Screen flow

b. Screen Details

#	Feature	Screen	Description
1	Home	Home (/)	Landing page with hero, suggested recipes and shortcuts; first visit asks the visitor to choose a diet type.
2	Account	Login (/dang-nhap)	E-mail and password sign-in; links to register and forgot password.
3	Account	Register (/dang-ky)	Create an account with full name, e-mail, password and agreement to terms.
4	Account	Forgot password (/quen-mat-khau)	Request a 6-digit OTP by e-mail and verify it.
5	Account	Reset password (/dat-lai-mat-khau)	Set a new password with the reset token obtained after OTP verification.
6	AI assistant	Chat widget	Floating assistant: nutrition chat, image scan, weekly menu, YouTube video summary, BMI shortcut, member history.
7	AI assistant	Meal plan dialog	Form for body data, goal, activity, diet, allergies, pantry and number of days; shows the generated plan.
8	Food knowledge	Recipes (/cong-thuc)	Search and filter recipes by category and diet; recipe detail with ingredients, steps and nutrition calculation.
9	Food knowledge	Ingredient dictionary (/tu-dien-chay)	Search ingredients, view nutrition per 100 g, origin, cooking notes and image; request AI lookup for unknown ingredients.
10	Food knowledge	Food swap (/doi-mon-chay)	Search a non-vegetarian ingredient and see vegetarian alternatives with usage tips.
11	Food knowledge	Food scanner (/kiem-tra-mon)	Upload a dish or label photo, or type ingredients, and check it against the selected diet.
12	Restaurants	Restaurant map (/quan-chay)	Map and list of vegetarian restaurants / ingredient shops near a location; filters, detail, reviews, suggest a place.
13	Community	Community (/cong-dong)	Feed of BLOG and VIDEO posts; composer, comments, votes and report action.

#	Feature	Screen	Description
14	Personal	Journal (/nhat-ky)	Daily food diary by meal group with nutrient totals, BMI and weekly calorie history; weekly plan tab.
15	Personal	Profile (/ho-so)	Account information, password change, health profile, diet type and health goal.
16	Administration	Dashboard (/admin?tab=dashboard)	Counters for members, content, AI moderation and reports; urgent items; system health.
17	Administration	Members (tab=members)	Search members, view their content, lock / unlock, delete and restore accounts.
18	Administration	Posts & videos (tab=posts)	Filter posts by type and status, edit, hide, restore or delete.
19	Administration	Comments (tab=comments)	Filter comments, hide, restore or delete.
20	Administration	Categories (tab=categories)	Create, rename and delete recipe / post categories.
21	Administration	AI moderation (tab=ai-moderation)	Queue of content hidden or flagged by AI; approve or remove.
22	Administration	AI monitor (tab=ai-monitor)	AI model status, log list and statistics, manual intervention on failed logs.
23	Administration	AI management (tab=ai-management)	AI usage and logs by feature, guest quota lookup / reset and chatbot cache clearing.
24	Administration	Reports (tab=reports)	List violation reports by status; resolve (optionally hide content) or dismiss.

c. User Authorization

X = allowed. 'Own' = only records owned by the current user. 'Quota' = allowed within the shared daily guest AI quota. Permissions are enforced by the back-end (authenticate + authorize middleware and ownership checks), not only by the UI.

Screen / Activity	Guest	Member	Admin
Home, Recipes, Ingredient dictionary, Food swap	X	X	X
View public content	X	X	X
Request AI lookup of a new ingredient	Quota	X	X
AI assistant (chat, meal plan, scan, video summary)	Quota	X	X
View own AI history		Own	Own
Submit YouTube video summary		X	X
Food scanner	Quota	X	X
Restaurant map	X	X	X
Write review / suggest a place		X	X
Community	X	X	X
Create post, comment, vote, report		X	X
Edit / delete post or comment		Own	X
Journal and weekly plan		Own	Own
Apply AI plan / record day into diary		Own	Own
Profile and notifications		Own	Own
Admin dashboard and system health			X
Members management (lock, delete, restore)			X
Posts, comments and categories management			X
AI moderation, AI monitor, AI management			X
Violation reports (resolve / dismiss)			X

In which:

- **Guest:** unauthenticated visitor identified for AI quota by a signed HttpOnly guest cookie; not stored as a role value.
- **Member:** users.role = 2 (USER) with status ACTIVE. Locked or deleted accounts cannot use protected APIs.
- **Admin:** users.role = 1 (ADMIN). Can moderate content and manage accounts but does not get an endpoint to read every member's health records.

5. Entity Relationship Diagram

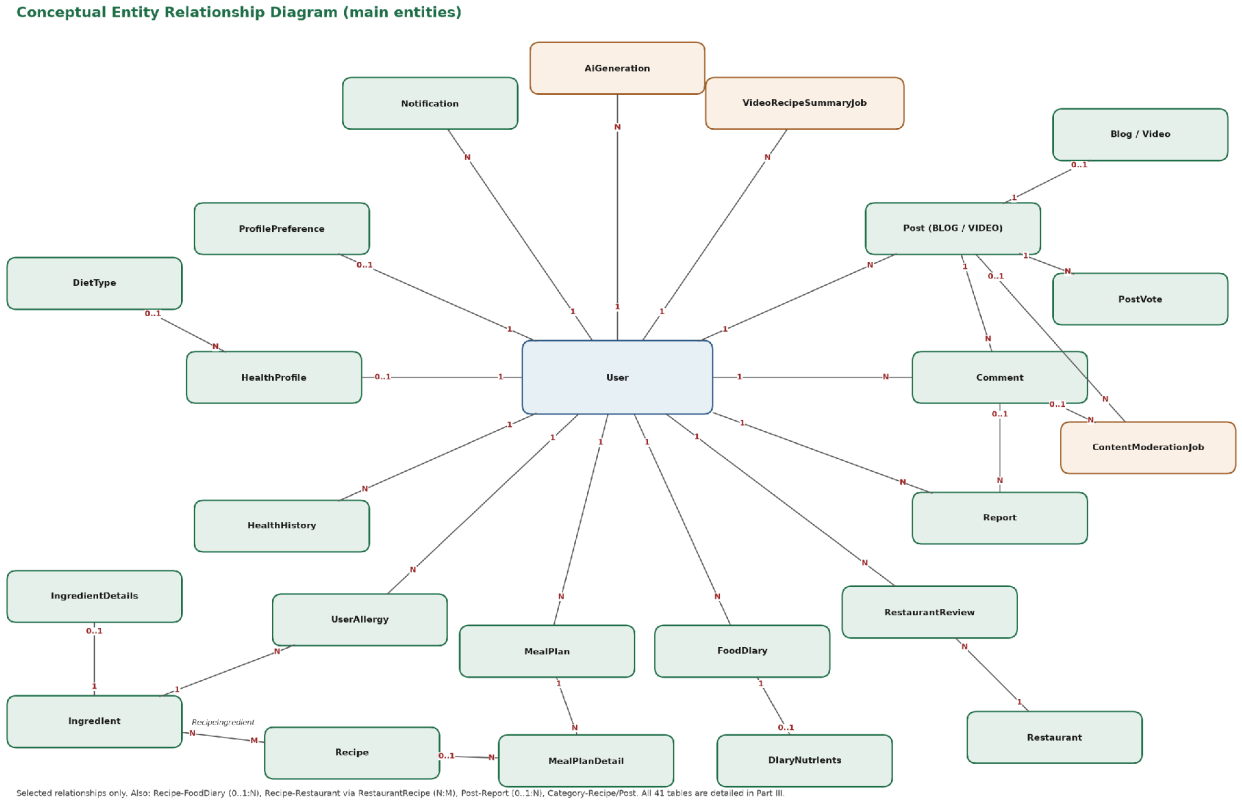


Figure 6. Conceptual entity relationship diagram

Entities Description

#	Entity	Description
1	User	Account with credentials, role (ADMIN / USER), status (ACTIVE / LOCKED / DELETED) and token version.
2	PasswordResetOtp	One active hashed 6-digit e-mail OTP per user with attempts and expiry.
3	PasswordResetToken	Hashed single-use reset token issued after OTP verification.
4	GuestQuota	Daily AI usage counter of a guest identity (Vietnam calendar day).
5	DietType	Diet catalogue: vegan, lacto, ovo, lacto-ovo, Buddhist, etc.
6	HealthProfile	Height, weight, birth date, gender, diet type and health goal of a member.
7	HealthHistory	Recorded weight and BMI history.
8	UserAllergy	Ingredients a member is allergic to or excludes.
9	ProfilePreference	Calorie goal and protein / carbs / fat / fiber targets.
10	Category	Category used by recipes and community posts.
11	Ingredient	Ingredient with calories and macro-nutrients per 100 g.
12	IngredientDetails / IngredientImage	Descriptive details (category, origin, plant flag, cooking notes, aliases) and a sourced image.
13	IngredientAlternative	Vegetarian substitution suggestions for a non-vegetarian ingredient.
14	LiquidIngredientDensity	Grams per millilitre used to convert liquid quantities.

#	Entity	Description
15	IngredientLookupJob	Background AI job that enriches an unknown ingredient.
16	Recipe / RecipeIngredient	Vegetarian recipe and its ingredient quantities.
17	MealPlan / MealPlanDetail	A member's weekly plan and its meal slots (recipe or custom food with nutrients).
18	FoodDiary / DiaryNutrients	Daily food journal entry and its nutrient snapshot.
19	Restaurant / RestaurantDetails	Vegetarian restaurant or ingredient shop with location and extra details.
20	RestaurantReview	Member star rating (1-5) and text review.
21	RestaurantRecipe / RestaurantMenuPrice	Dishes served by a restaurant and optional price.
22	Post / Blog / Video	Community publication with BLOG or VIDEO subtype.
23	PostCategory / PostAttachment	Post categories and uploaded media.
24	Comment / PostVote	Flat comments and one vote per member per post.
25	ContentModerationJob	Versioned AI moderation job for a post or comment.
26	VideoRecipeSummaryJob / VideoRecipeSummary	YouTube recipe extraction job and its structured result.
27	AiGeneration	A member's AI request and response (CHAT, MEAL_PLAN, FOOD_SCAN).
28	AiLog	AI model / action / status log used for monitoring.
29	Report	Violation report on a post or comment.
30	Notification	Message shown to a member (e.g. report outcome, post removal).

6. State transition diagram

The main entities with a lifecycle are community posts (and comments), violation reports and the background jobs.

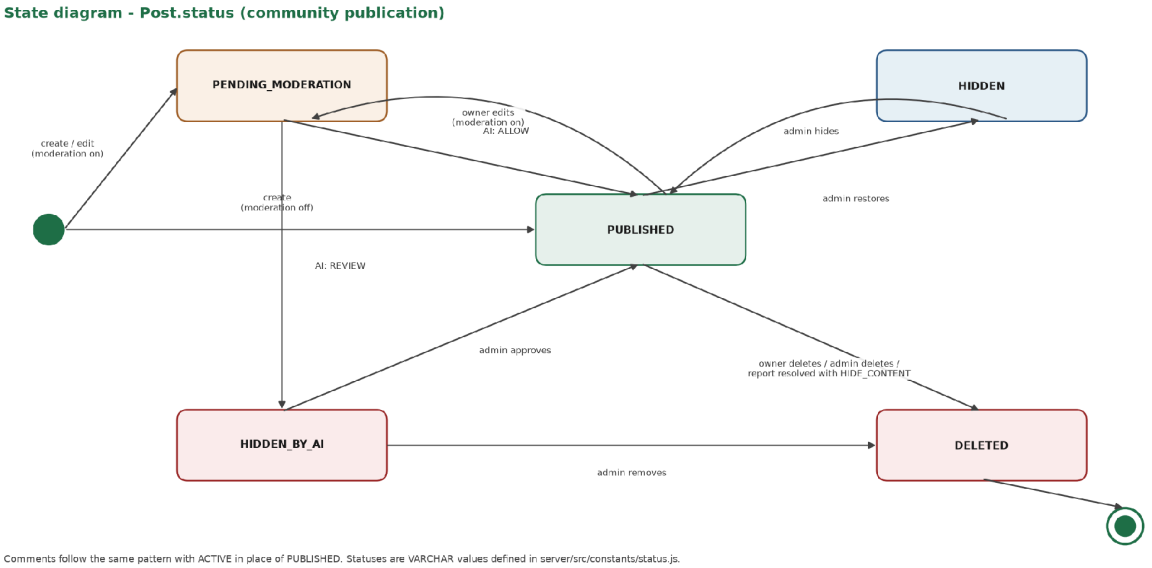
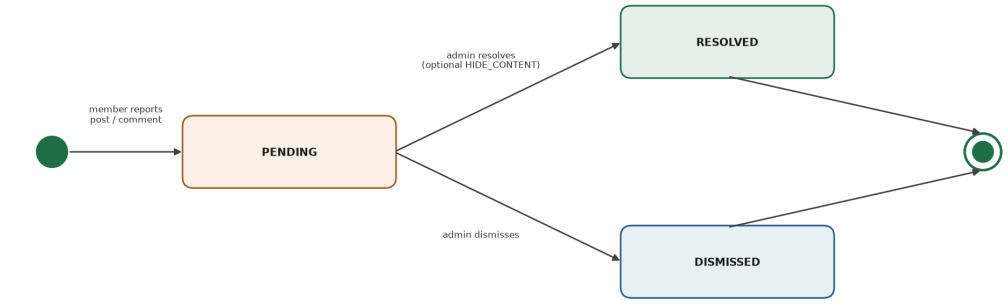


Figure 7. State transition of Post.status

Status	Meaning
PENDING_MODERATION	Created or edited while automated moderation is enabled; waiting for the AI worker.
PUBLISHED	Visible in the public feed (comments use ACTIVE).
HIDDEN_BY_AI	AI returned REVIEW; hidden until an administrator approves or removes it.
HIDDEN	Hidden by an administrator; can be restored.
DELETED	Soft-deleted by the owner, an administrator or a resolved report.

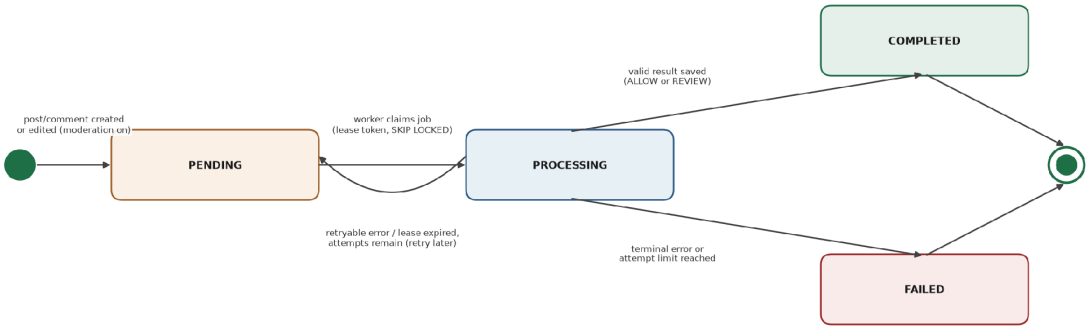
State diagram - Report.status



The reporter receives a notification for both outcomes.

Figure 8. State transition of Report.status

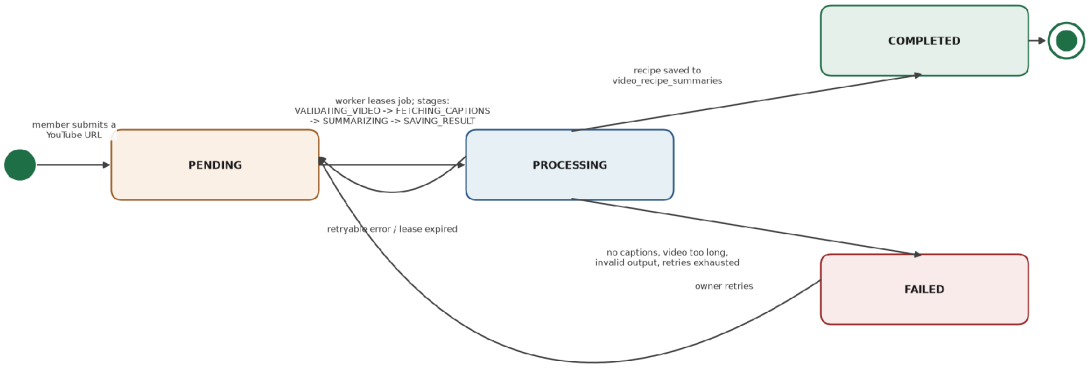
State diagram - ContentModerationJob.status



Content state is separate: a result is applied only when the job's content version still matches the post/comment.

Figure 9. State transition of ContentModerationJob.status

State diagram - VideoRecipeSummaryJob.status (with stage field)



Stages FETCHING_AUDIO and TRANSCRIBING exist in the schema but are not used by the current captions-only pipeline.

Figure 10. State transition of VideoRecipeSummaryJob.status

II. Functional Requirements

1. Account and Access

a. Register

- **Function trigger:** Header link 'Đăng ký' or the register tab on the login page (/dang-ky).
- **Function description:** A guest creates a member account. Actor: Guest. The new account receives role USER (2) and status ACTIVE.
- **Screen layout:**

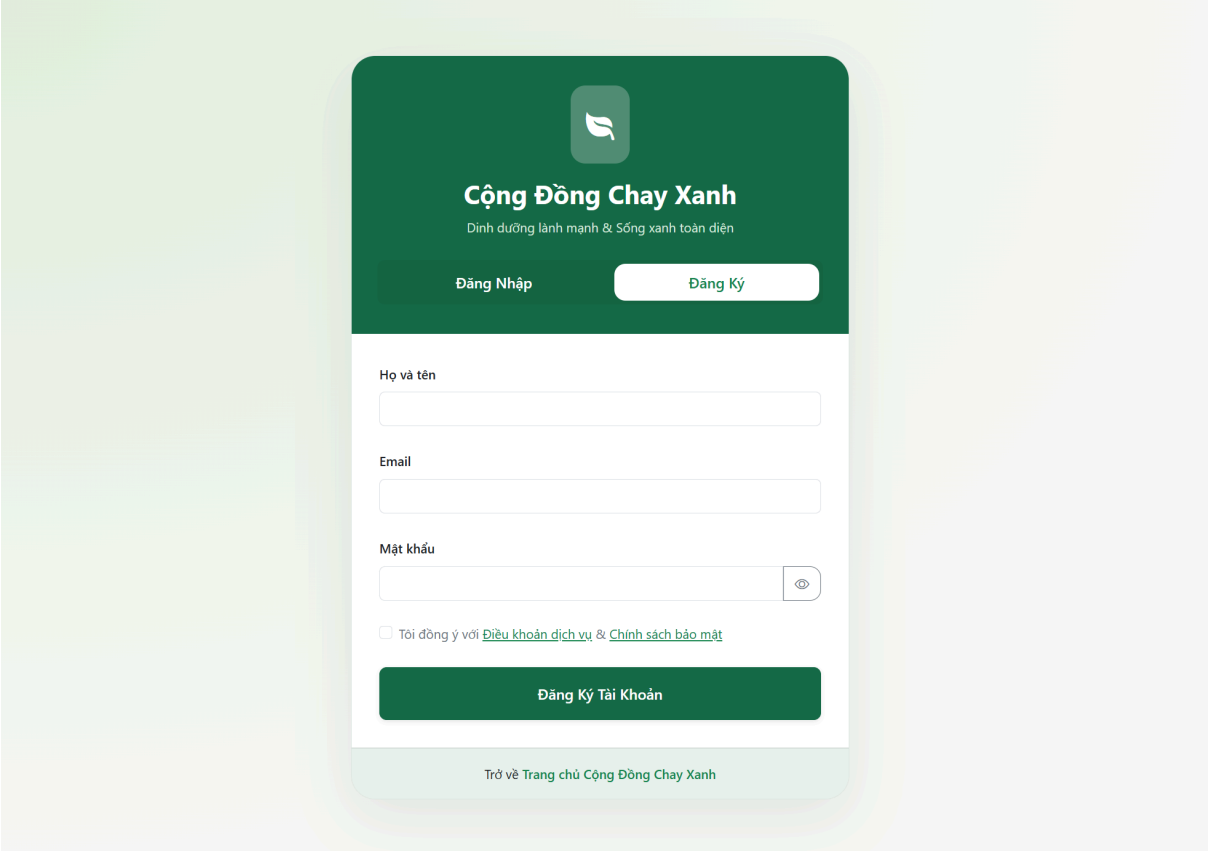


Figure 11. Register screen

- **Function details:**
  - **Data:** full name (required, <= 255 characters), e-mail (valid format, <= 255), password (8-128 characters), agreement checkbox.
  - **Normal flow:** the API validates the body with Zod, checks that the e-mail is not used, hashes the password with bcrypt and creates the users record, then signs the user in.
  - **Abnormal flow:** invalid fields are shown under each input; an existing e-mail returns 409 with an explicit errorCode.
  - **Rules:** role cannot be chosen by the client; it is always USER.

b. Login and logout

- **Function trigger:** Header link 'Đăng nhập' (/dang-nhap); logout from the user menu.
- **Function description:** Authenticate with e-mail and password, keep the session with a JWT access token and refresh it when it expires. Actors: Member, Admin.

- **Screen layout:**

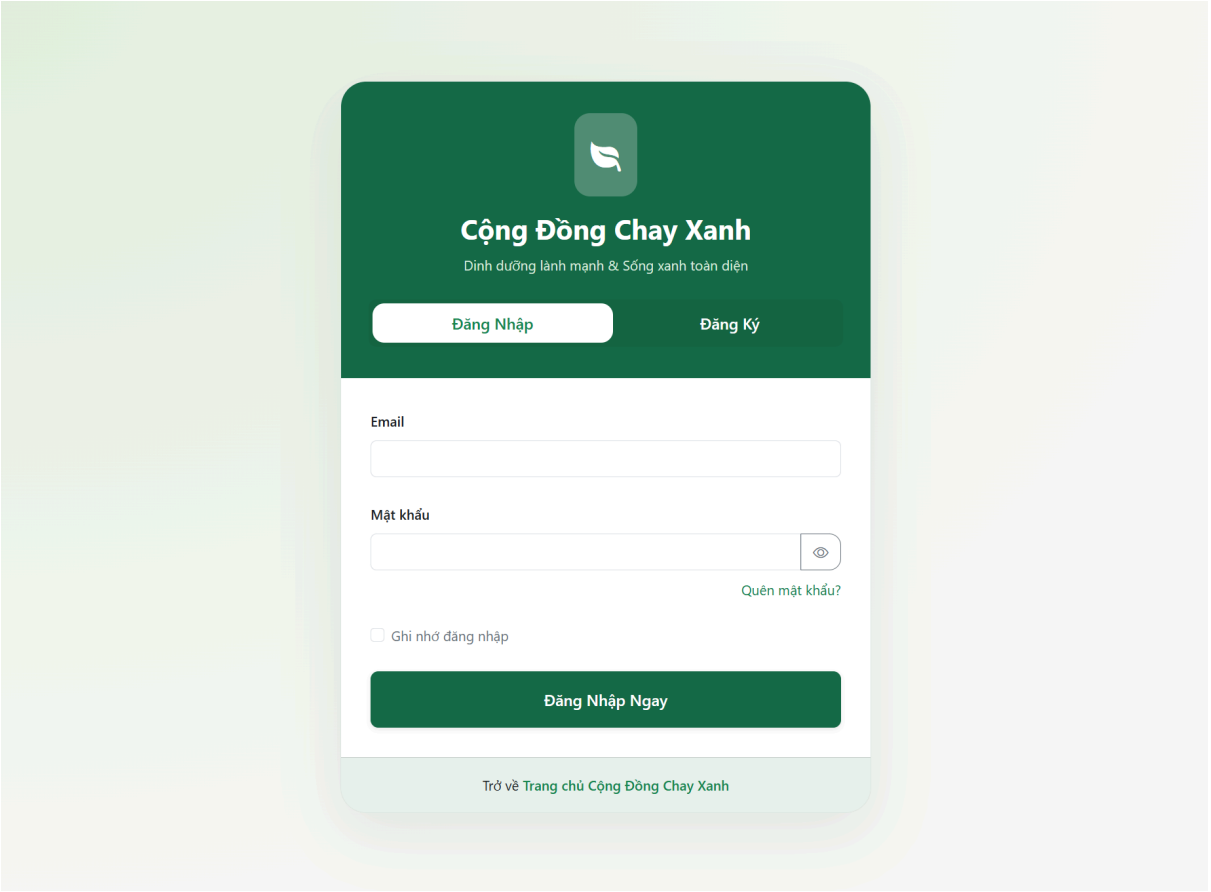


Figure 12. Login screen

- **Function details:**
  - **Data:** e-mail and password are required (password <= 128 characters); optional 'remember me'.
  - **Normal flow:** valid credentials return the token and the account; ADMIN users are redirected to /admin, members to the previous page or home.
  - **Abnormal flow:** wrong credentials return 401 with a generic message; LOCKED or DELETED accounts are refused; an invalid or expired token forces a new login.
  - **Rules:** changing or resetting the password increases token_version, so tokens issued before become invalid; repeated login attempts are rate-limited.

c. Forgot and reset password

- **Function trigger:** Link 'Quên mật khẩu' on the login page (/quen-mat-khau), then /dat-lai-mat-khau.
- **Function description:** Recover an account through a 6-digit OTP sent by e-mail, then set a new password. Actor: Guest.

- **Screen layout:**

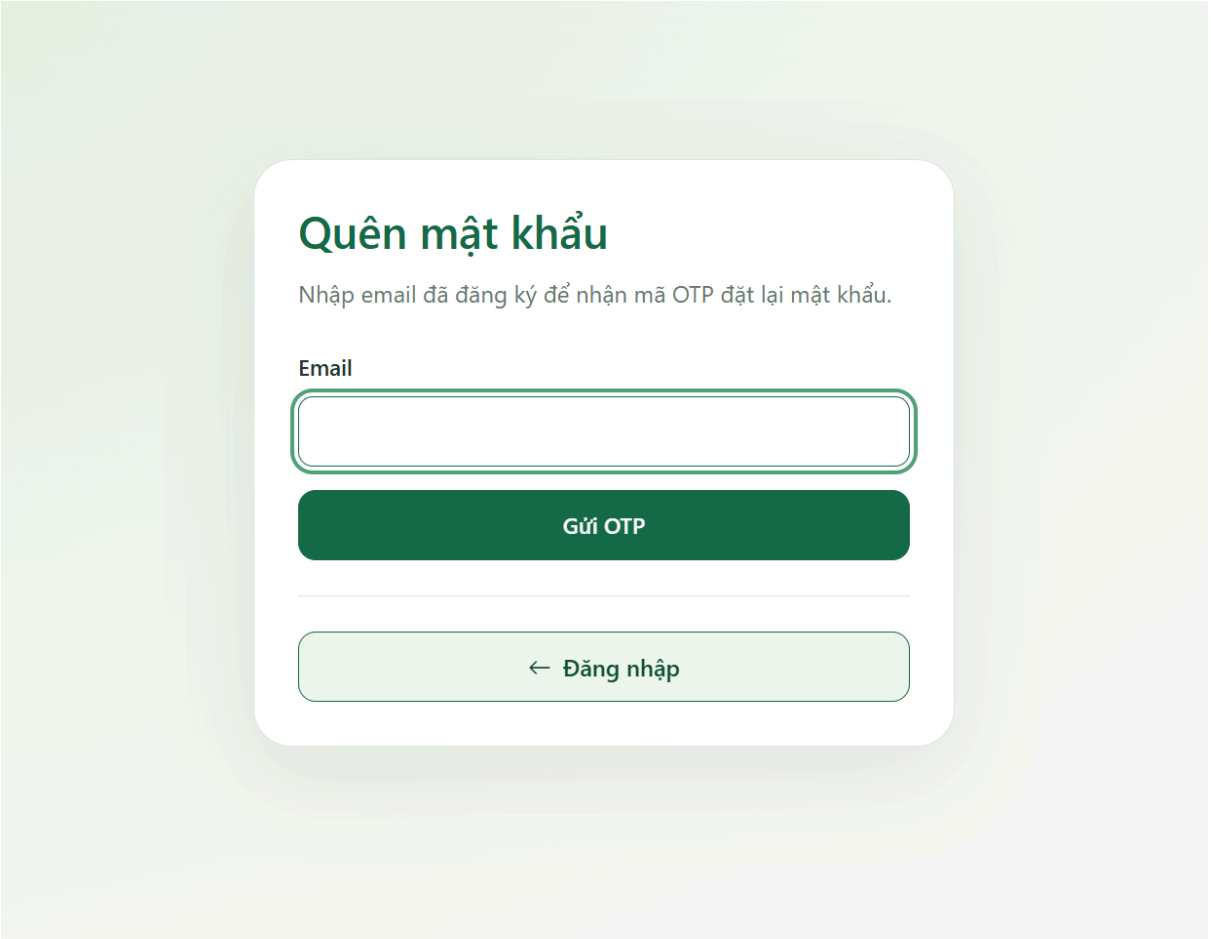


Figure 13. Forgot password screen

- **Function details:**
  - **Data:** e-mail <= 255; OTP exactly 6 digits; reset token 64 hex characters; new password 8-128 characters and <= 72 bytes.
  - **Normal flow:** the API sends the OTP by SMTP and stores only its hash (password_reset_otps); a correct OTP returns a single-use reset token (stored hashed); submitting a new password updates the hash and token version.
  - **Abnormal flow:** wrong, expired, consumed or over-attempt codes and tokens are rejected; SMTP failure is reported, not shown as success.
  - **Rules:** OTP and token lifetime are configurable (default 15 minutes); a token can be used only once.

2. AI Nutrition Assistant

a. Nutrition chat

- **Function trigger:** Click 'Trợ lý ChayXanh' at the bottom-right corner of any site page.
- **Function description:** Ask nutrition and vegetarian cooking questions; answers are streamed token by token. Actors: Guest (quota), Member, Admin.

- **Screen layout:**

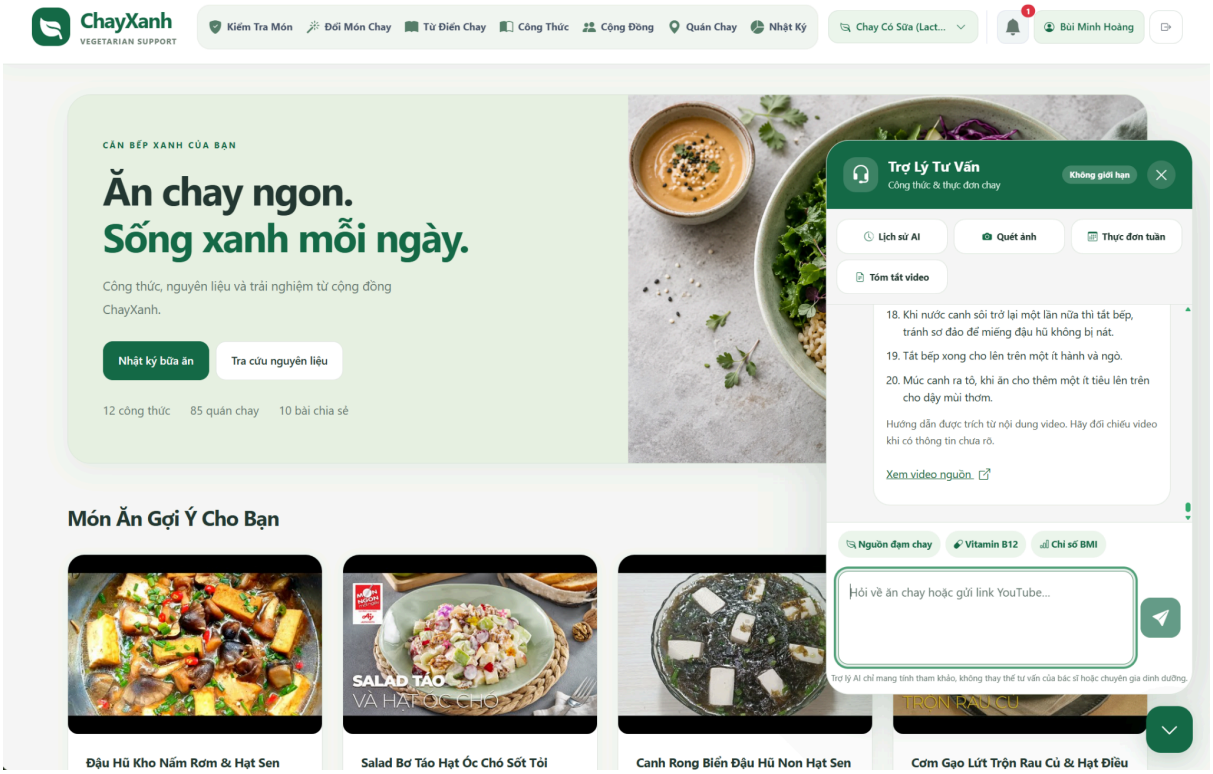


Figure 14. AI assistant widget

- **Function details:**
  - **Data:** message 1-5000 characters; up to 10 previous messages (<= 8000 characters each) as context; diet type from the user's choice.
  - **Normal flow:** the API checks identity / guest quota, calls the AI service over SSE and relays tokens; a member's conversation is saved in ai_generations (type CHAT) and listed in history.
  - **Abnormal flow:** provider errors or a broken stream end with an explicit error message; nothing is saved as COMPLETED.
  - **Rules:** answers are advisory; a member can only read his / her own history.

b. Generate weekly meal plan

- **Function trigger:** AI assistant > 'Thực đơn tuần'.
- **Function description:** Generate a vegetarian menu for 1-7 days that respects the diet type and allergies. Actors: Guest (quota), Member.

- **Screen layout:**

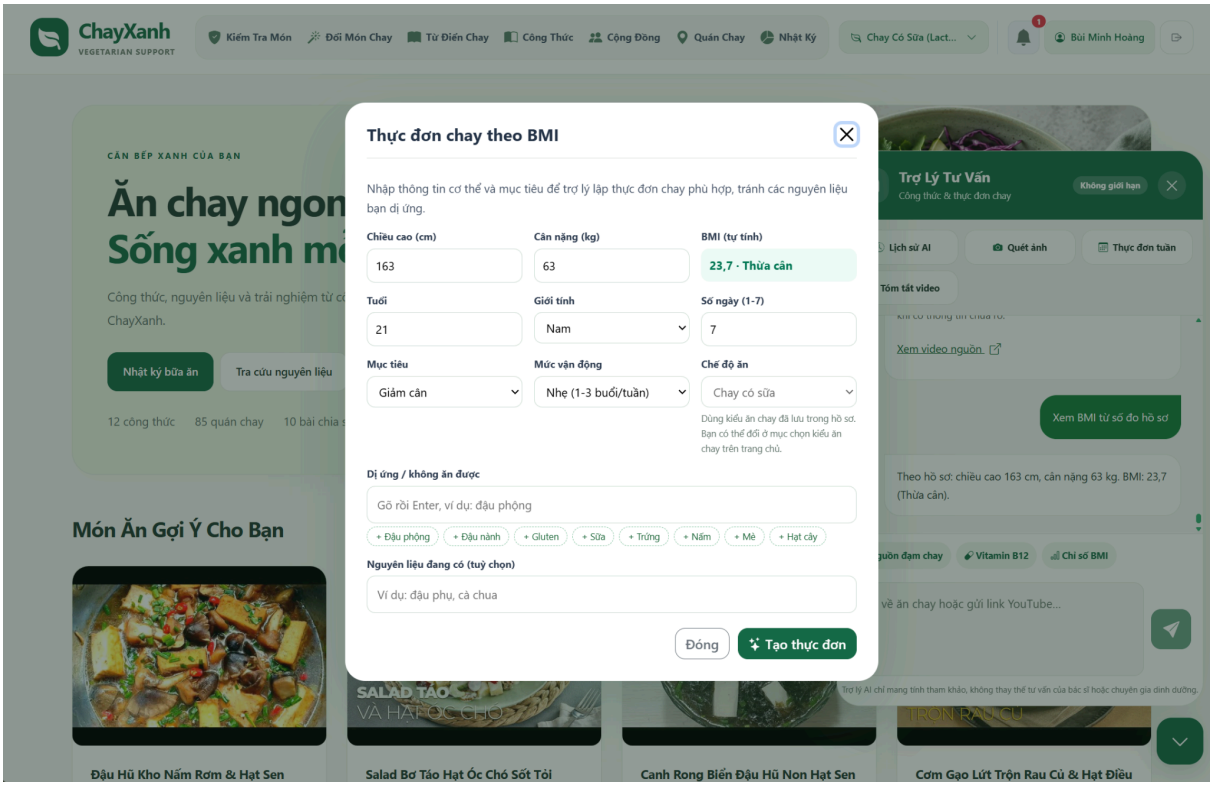


Figure 15. Meal plan dialog

- **Function details:**
  - **Data:** height 50-260 cm, weight 20-300 kg, age 10-120, gender, goal, activity level, days 1-7 (default 7), up to 100 allergies and 100 pantry items.
  - **Normal flow:** see Business Process 3.1: the AI service computes the energy target and returns breakfast, lunch, dinner and optional snack per day; the result is stored as a MEAL_PLAN generation for members.
  - **Abnormal flow:** if diet / allergy / day-count rules still fail after 3 attempts the request fails and the guest turn is released.
  - **Rules:** calorie and macro targets are soft; diet and allergy exclusions are hard rules. Generating does not change the journal.

c. Guest AI quota

- **Function trigger:** Any AI request by a guest (chat, meal plan, scan, new ingredient lookup).
- **Function description:** Limit AI usage of unauthenticated visitors with one shared daily counter.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
  - **Data:** guest identity from a signed HttpOnly cookie; guest_quota(guest_id, quota_date, used_count, reservation_revision).
  - **Normal flow:** a turn is reserved before calling the AI; it is released if the generation fails.
  - **Abnormal flow:** when the limit (GUEST_DAILY_LIMIT, default 3) is reached the API returns 429 AI_QUOTA_EXCEEDED with the reset time and the UI asks the visitor to sign in.

- **Rules:** the day resets at midnight Vietnam time (UTC+7); members are not limited by this quota; an administrator can look up or reset a guest quota.

d. YouTube video recipe summary

- **Function trigger:** AI assistant > 'Tóm tắt video', or paste a YouTube link in the chat.
- **Function description:** Extract a structured recipe (ingredients, steps) from a cooking video using its Vietnamese captions. Actor: Member.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
  - **Data:** HTTPS YouTube URL of a public, non-live video up to 11 minutes (660 s); no playlist.
  - **Normal flow:** a video_recipe_summary_jobs record is queued; the worker validates the video, fetches captions (manual first, then auto-generated Vietnamese), summarises with the LLM and saves video_recipe_summaries.
  - **Abnormal flow:** no captions, too long or invalid output marks the job FAILED; the owner can retry it.
  - **Rules:** one active job per member; the same video and pipeline version is reused instead of re-processing. The UI is part of the chat widget (Figure of the AI assistant).

3. Food Knowledge

a. Recipe catalog

- **Function trigger:** Header menu 'Công thức' (/cong-thuc).
- **Function description:** Browse vegetarian recipes, filter by category and diet type, and open details with ingredients, instructions and calculated nutrition. Actors: all.
- **Screen layout:**

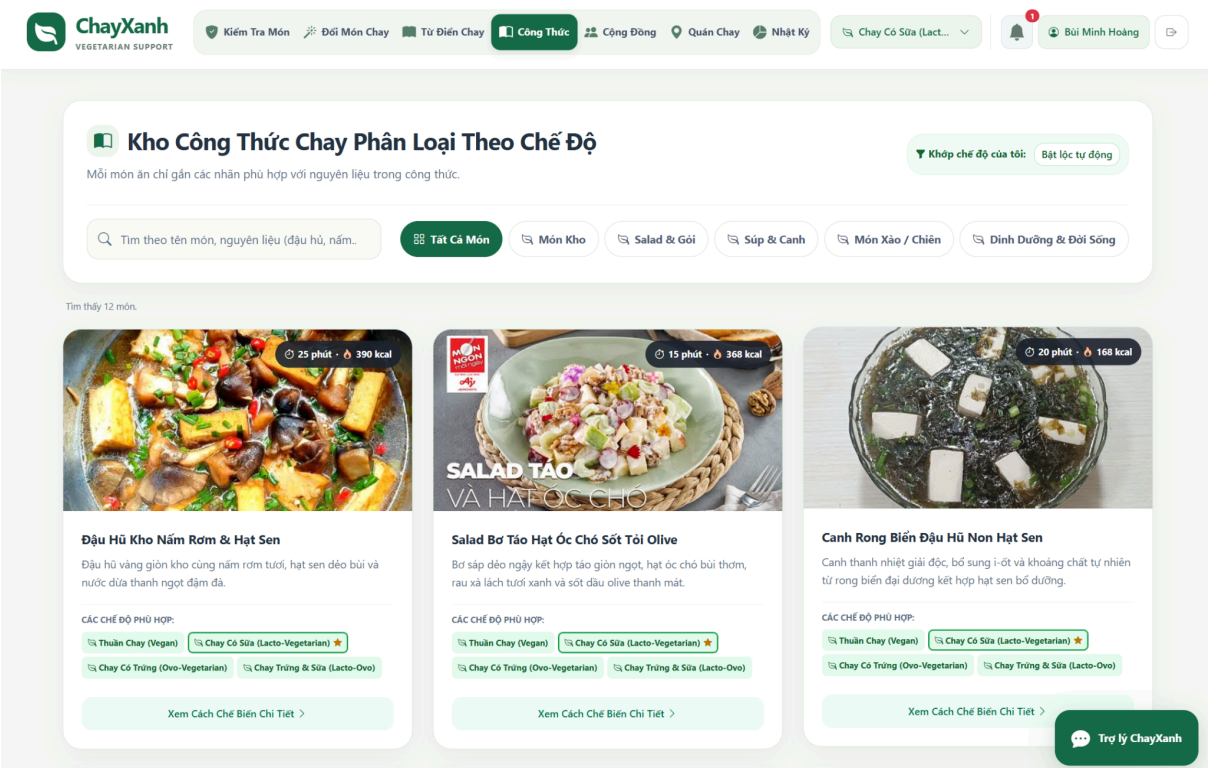


Figure 16. Recipes screen

- **Function details:**

- **Data:** recipes, recipe_ingredients (quantity, unit), ingredients (per 100 g values), liquid_ingredient_densities.
- **Normal flow:** the list shows image, preparation time, calories and the diet labels that suit the recipe; detail calculates calories and macros from stored quantities.
- **Abnormal flow:** missing nutrition data is shown as unavailable, not as zero.
- **Rules:** a member can add a recipe to the journal or weekly plan from the journal screens.

b. Ingredient dictionary and AI lookup

- **Function trigger:** Header menu 'Từ điển chay' (/tu-dien-chay).
- **Function description:** Search ingredients and view nutrition per 100 g, origin, cooking notes and a sourced image; request AI enrichment when an ingredient is unknown. Actors: all (lookup uses guest quota).
- **Screen layout:**

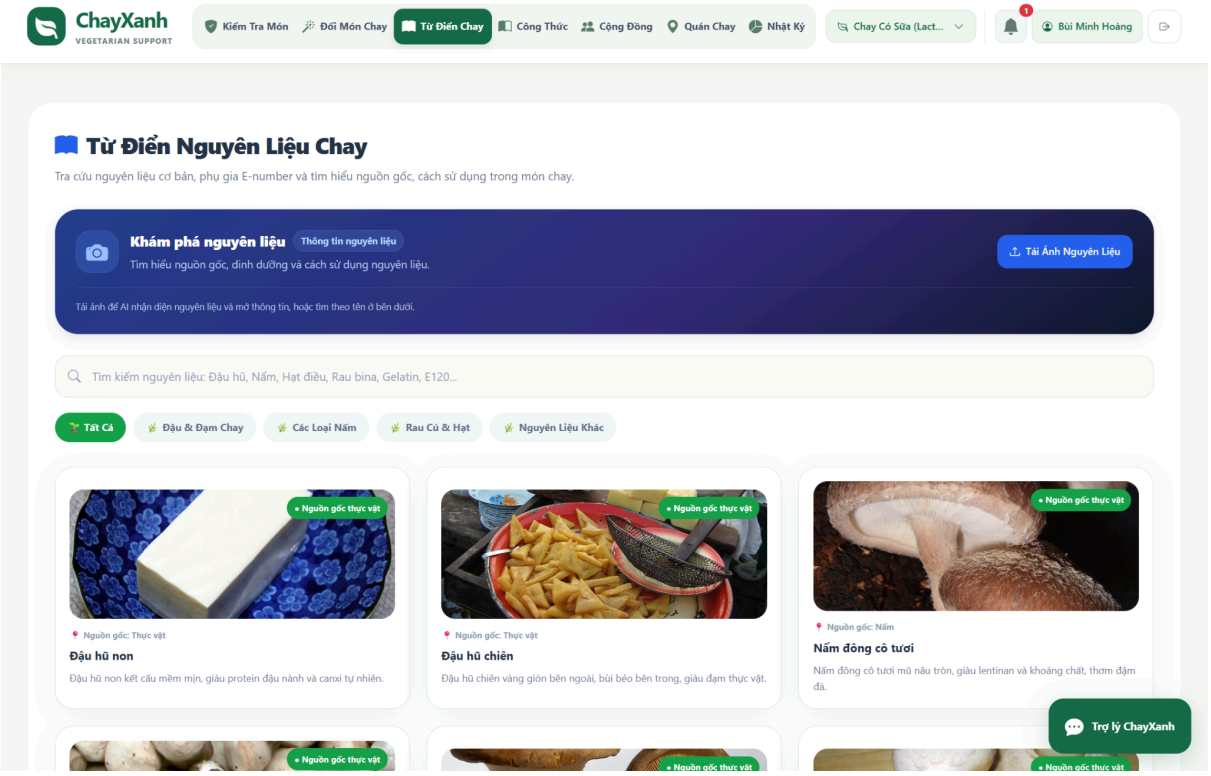


Figure 17. Ingredient dictionary screen

- **Function details:**
  - **Data:** ingredient name (normalised), optional source photo; ingredient_details, ingredient_images (with source and license).
  - **Normal flow:** an existing detailed ingredient is returned immediately; otherwise an ingredient_lookup_jobs record is queued (202) and the page polls the job until COMPLETED.
  - **Abnormal flow:** a FAILED job shows the error; no placeholder ingredient is created.
  - **Rules:** duplicate active lookups for the same name are merged; existing ingredients do not consume a guest turn.

c. Food swap

- **Function trigger:** Header menu 'Đổi món chay' (/doi-mon-chay).

- **Function description:** Search a non-vegetarian ingredient (e.g. fish sauce, gelatin) and see vegetarian alternatives with their role and tips. Actors: all.
- **Screen layout:**

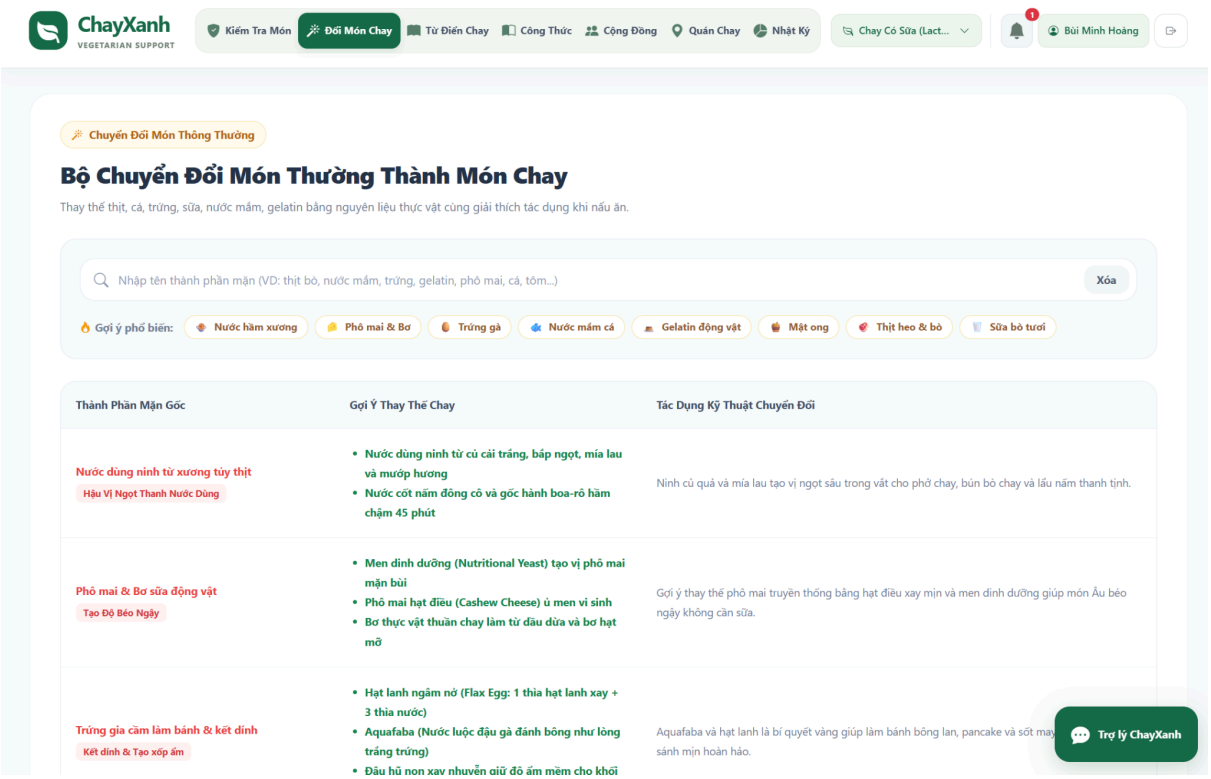


Figure 18. Food swap screen

- **Function details:**
  - **Data:** ingredient_alternatives: original, role, alternatives (JSON), description, keywords.
  - **Normal flow:** keyword search and quick chips filter the curated list.
  - **Abnormal flow:** no match shows an empty state.
  - **Rules:** content is read-only for users.

d. Food scanner

- **Function trigger:** Header menu 'Kiểm tra món' (/kiem-tra-mon) or 'Quét ảnh' in the AI assistant.
- **Function description:** Check a dish photo, a product label photo or a typed ingredient list against the selected diet. Actors: all (guest quota).

- **Screen layout:**

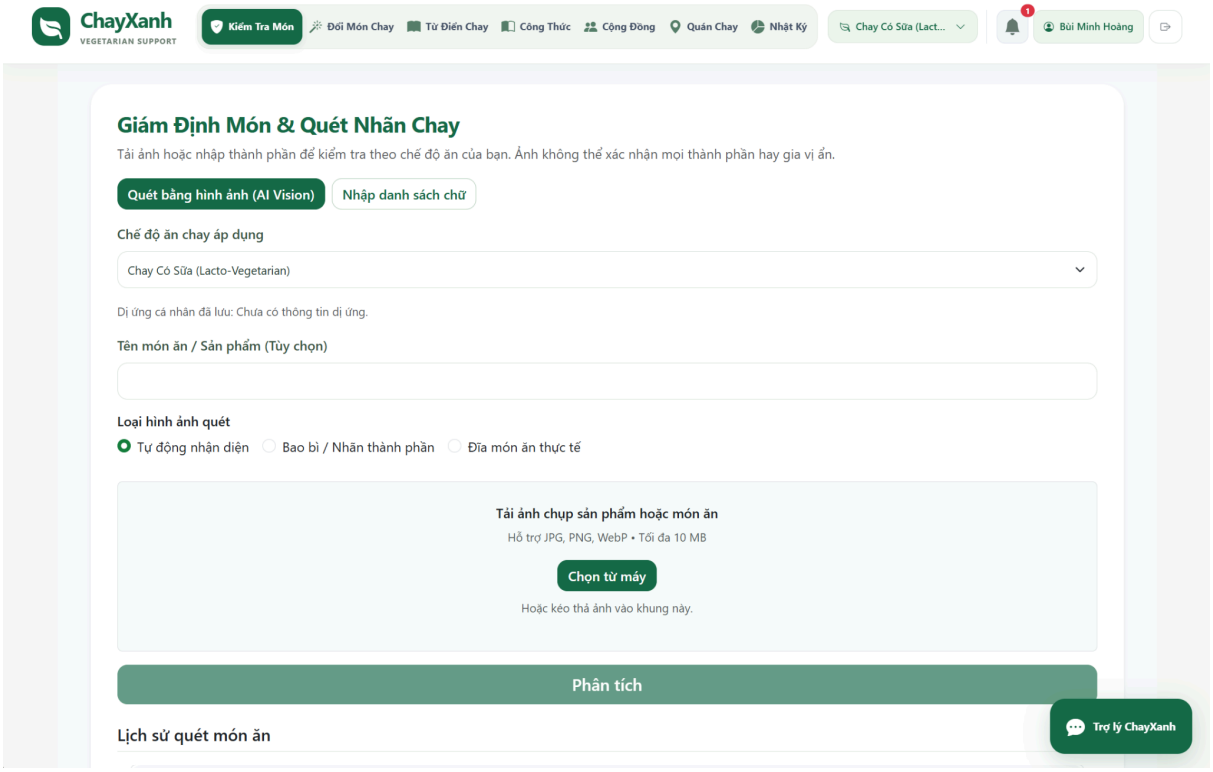


Figure 19. Food scanner screen

- **Function details:**
  - **Data:** JPEG / PNG / WebP image up to 10 MB with matching file signature, diet type, mode AUTO / LABEL / DISH, optional dish name and up to 50 allergies.
  - **Normal flow:** the AI returns clear or warning, detected type, summary, findings with severity, identified ingredients, checklist, swap suggestions and an estimated nutrition block.
  - **Abnormal flow:** invalid images are rejected; provider or schema errors are shown as errors, never as a successful scan.
  - **Rules:** Buddhist diet also checks the five pungent vegetables; a 'clear' result cannot contain a violation.

4. Restaurant Discovery

a. Search restaurants on the map

- **Function trigger:** Header menu 'Quán chay' (/quan-chay).
- **Function description:** Find vegetarian restaurants or ingredient shops near the user's GPS position or a searched place, shown on a Leaflet map and as a list. Actors: all.

- **Screen layout:**

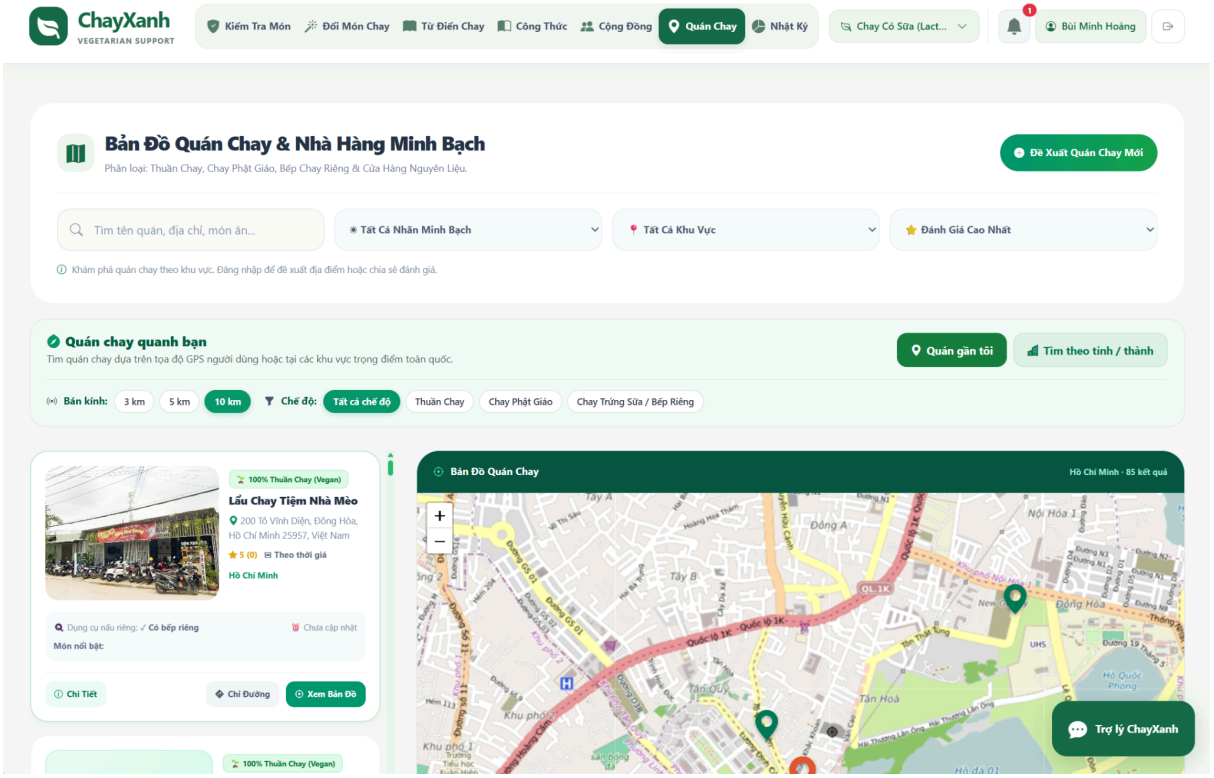


Figure 20. Restaurant map screen

- **Function details:**
  - **Data:** keyword, latitude (-90..90), longitude (-180..180), radius, diet type and place-type filters, sort order.
  - **Normal flow:** stored restaurants and ZIOMAP results are merged, filtered to vegetarian places and sorted by distance (geolib) or rating; selecting one shows its details and a Google Maps directions link.
  - **Abnormal flow:** denied geolocation, provider timeout or no matches are shown as such.
  - **Rules:** venues need a diet type; ingredient shops have no diet type.

b. Review a restaurant

- **Function trigger:** Restaurant detail > review form (login required).
- **Function description:** A member rates a place 1-5 stars with a text review. Actor: Member.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
  - **Data:** stars integer 1-5, content <= 1000 characters.
  - **Normal flow:** the review is saved in restaurant_reviews and the list and average are refreshed.
  - **Abnormal flow:** guests are asked to sign in; invalid values are rejected.
  - **Rules:** screen layout is part of the restaurant detail panel in the Restaurant map screen.

c. Suggest a new place

- **Function trigger:** Button 'Đề xuất quán chay mới' on the restaurant map (login required).
- **Function description:** A member proposes a restaurant or ingredient shop. Actor: Member.

- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
    - **Data:** name, address, city, type, diet type (venue only), coordinates, optional phone (<= 50), hours and price range (<= 100), separate kitchen flag, optional photo.
    - **Normal flow:** the place is saved with source = proposal and becomes visible on the map.
    - **Abnormal flow:** missing coordinates or invalid fields are rejected with field errors.
    - **Rules:** a venue must have a diet type; an ingredient shop must not.

5. Community

a. Publish blog and video posts

- **Function trigger:** Header menu 'Cộng đồng' (/cong-dong) > composer (login required).
- **Function description:** Members share blogs (text + up to 6 images / files) or cooking videos (YouTube link). Actors: Member; everyone can read the feed.
- **Screen layout:**

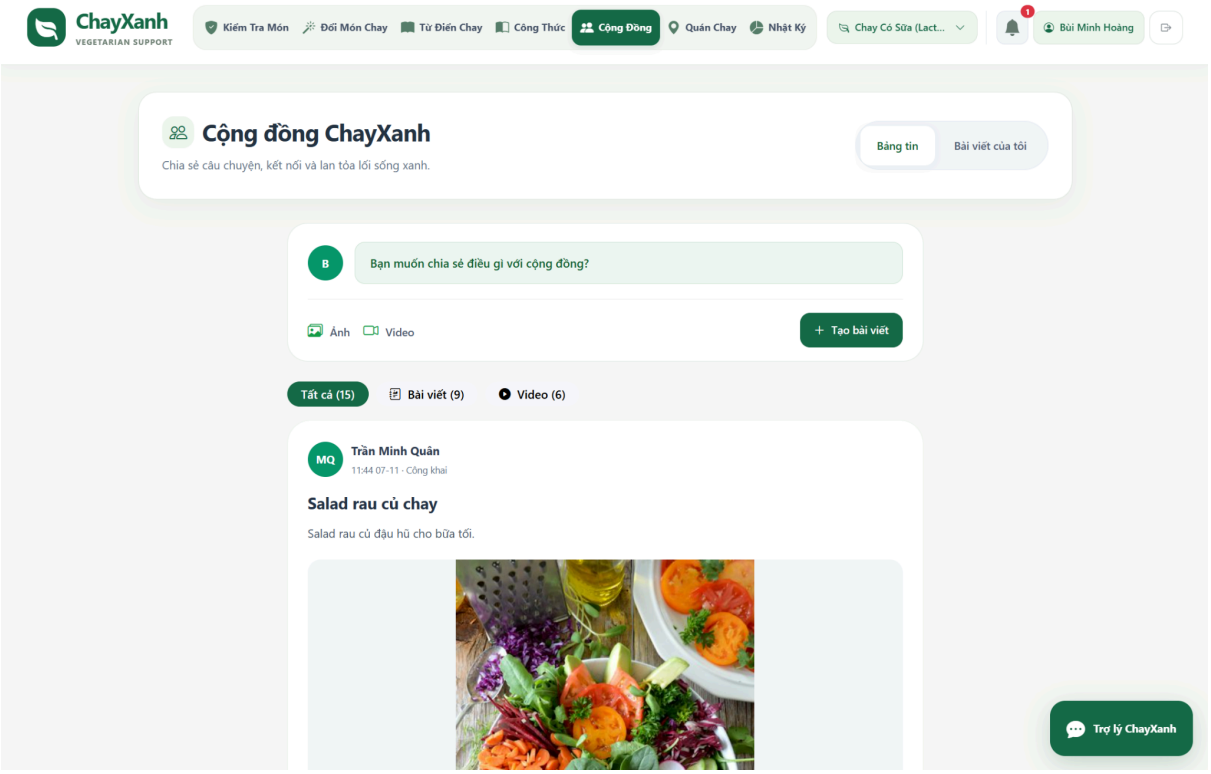


Figure 21. Community screen

- **Function details:**
    - **Data:** title <= 255, content <= 50000, categories, attachments uploaded to server/uploads (max 6), video URL for VIDEO posts.
    - **Normal flow:** a posts record and its blogs / videos subtype are created; the post is PUBLISHED or PENDING_MODERATION (see 3.2).
    - **Abnormal flow:** invalid media, foreign ownership or missing target is rejected.
    - **Rules:** owners can edit / delete their posts; editing re-triggers moderation when it is enabled.

b. Comment and vote

- **Function trigger:** Post card in the community feed (login required).

- **Function description:** Members comment on posts and vote UPVOTE / DOWNVOTE. Actor: Member.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
    - **Data:** comment content <= 5000 characters; vote_type UPVOTE or DOWNVOTE.
    - **Normal flow:** comments are flat (no replies); one vote per member per post, voting again changes or removes it.
    - **Abnormal flow:** commenting on a hidden or deleted post is refused.
    - **Rules:** comments are also moderated when moderation is enabled.

c. Report a violation

- **Function trigger:** Report action on a post or comment (login required).
- **Function description:** Members report inappropriate content with a reason. Actor: Member.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
    - **Data:** target post_id or comment_id and reason.
    - **Normal flow:** a reports record is created with status PENDING (see 3.3).
    - **Abnormal flow:** reporting content that is not published / active is refused.
    - **Rules:** the reporter is notified when the administrator resolves or dismisses the report.

d. Automated content moderation

- **Function trigger:** Creating or editing a post / comment while CONTENT_MODERATION_ENABLED = true.
- **Function description:** A background worker uses the AI service to inspect text and uploaded images and decide ALLOW or REVIEW.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
    - **Data:** content_moderation_jobs snapshot: title, content, image URLs, content_version.
    - **Normal flow:** worker polls every 2 s, leases the job, records the result and applies it only to the matching content version.
    - **Abnormal flow:** retryable errors are retried (default 3 attempts, 30 s delay); exhausted jobs become FAILED and appear in the admin queue.
    - **Rules:** the feature is disabled by default; REVIEW is a successful inspection (job COMPLETED), not a failure.

6. Personal Health

a. Profile and health profile

- **Function trigger:** User menu > 'Hồ sơ' (/ho-so).
- **Function description:** View and update account information, change password and maintain health data used by the planner and journal. Actor: Member.

- **Screen layout:**

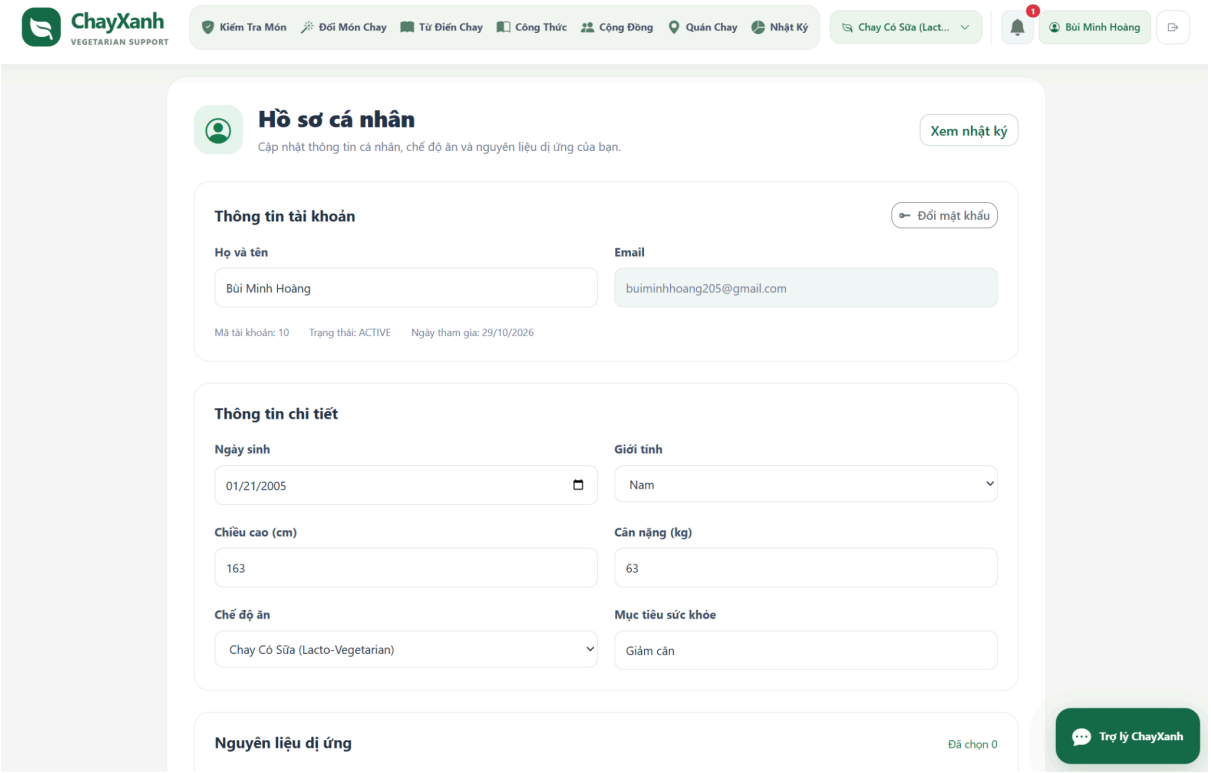


Figure 22. Profile screen

- **Function details:**
  - **Data:** height 50-250 cm, weight 10-400 kg, birth date, gender, diet type, health goal; allergies and nutrient goals.
  - **Normal flow:** data is saved to health_profiles, profile_preferences and user_allergies; BMI is computed from height and weight.
  - **Abnormal flow:** invalid values are rejected; missing data is shown as incomplete.
  - **Rules:** the API always uses the authenticated user id; another user's profile cannot be changed by sending an id.

b. Food journal

- **Function trigger:** User menu > 'Nhật ký' (/nhat-ky), tab 'Nhật ký trong ngày'.
- **Function description:** Record what was eaten per day and compare totals with personal targets. Actor: Member.

- **Screen layout:**

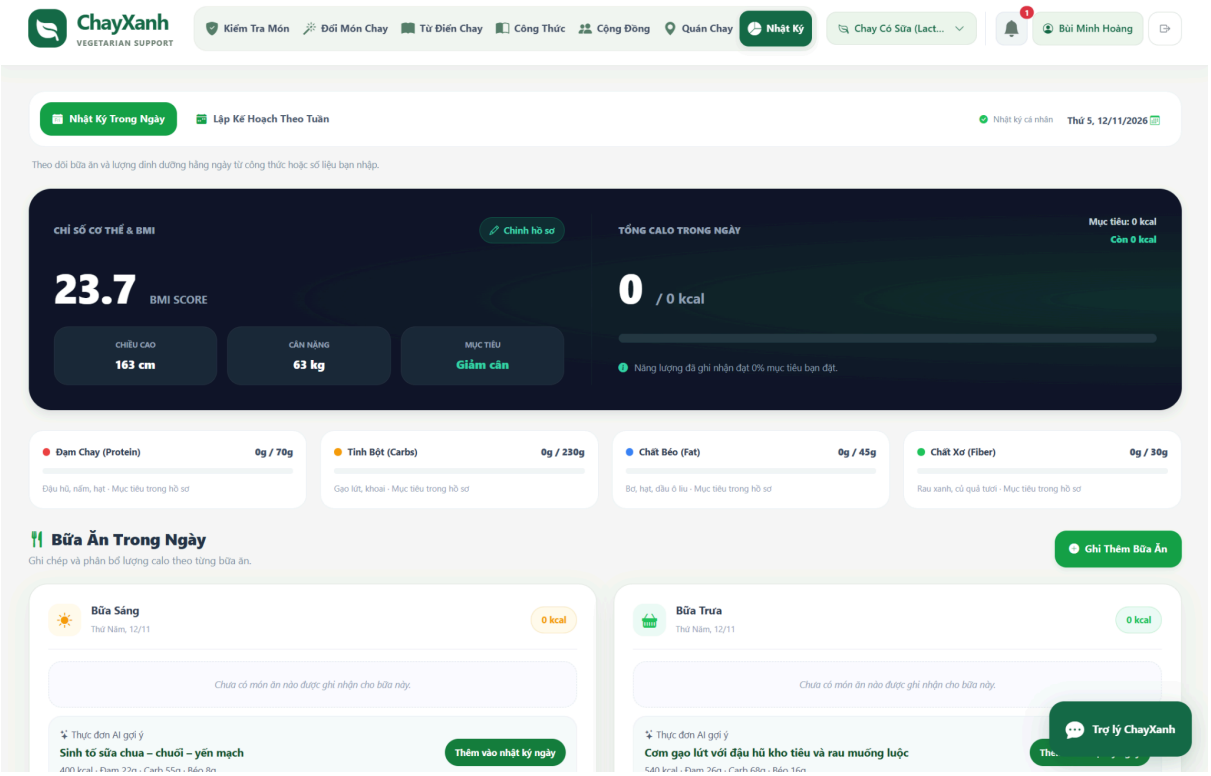


Figure 23. Journal screen (daily diary)

- **Function details:**
  - **Data:** date (YYYY-MM-DD), meal group BREAKFAST / LUNCH / DINNER / SNACK, recipe or custom food name, serving, calories 0-10000, protein / carbs / fat / fiber 0-1000.
  - **Normal flow:** entries are saved in food_diaries + diary_nutrients; the page shows BMI, calorie total versus goal, macro totals and weekly calorie history.
  - **Abnormal flow:** invalid or foreign entries are rejected.
  - **Rules:** deleting an entry updates the totals immediately.

c. Weekly plan and applying AI plans

- **Function trigger:** Journal > tab 'Lập kế hoạch theo tuần'.
- **Function description:** Plan meals for a week manually or apply a generated AI plan, then record a planned day into the diary. Actor: Member.

- **Screen layout:**

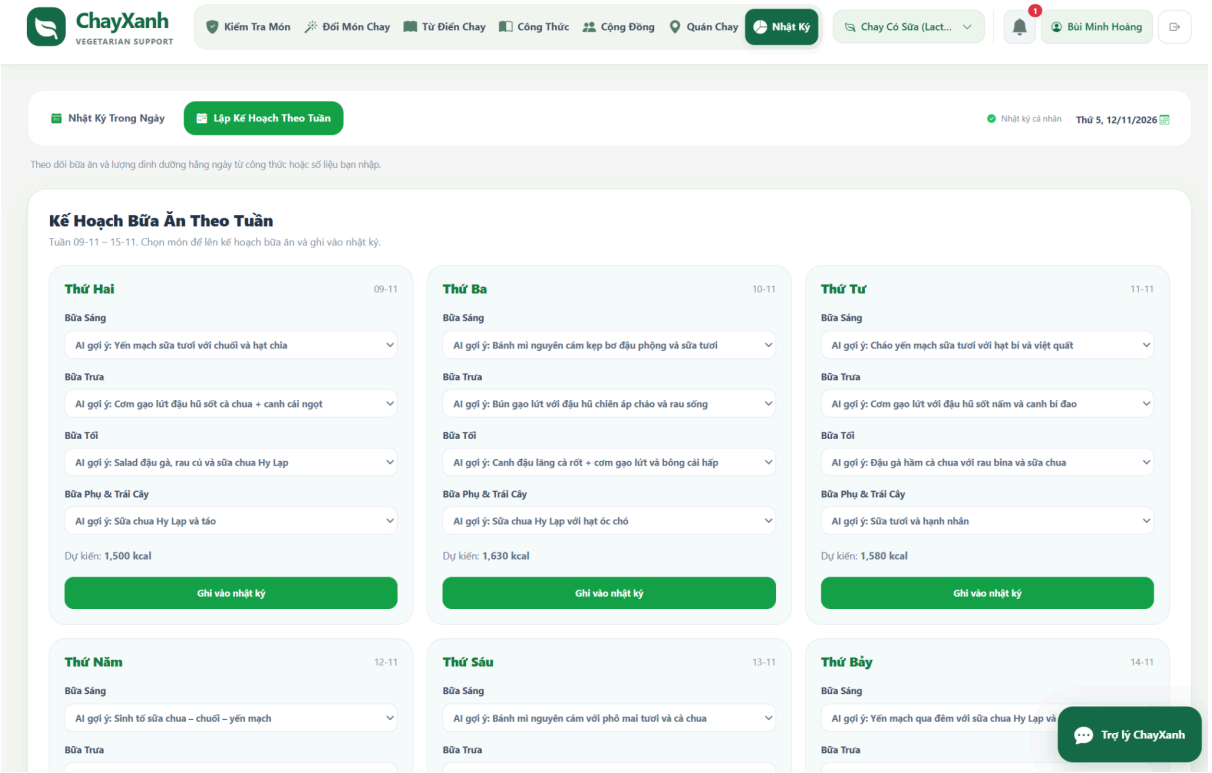


Figure 24. Journal - weekly plan tab

- **Function details:**
  - **Data:** generationId (UUID of an own COMPLETED MEAL_PLAN generation) and a date inside the target week.
  - **Normal flow:** the API replaces the Monday-Sunday week in meal_plans / meal_plan_details within one transaction; 'record day' copies that day's meals to the food diary.
  - **Abnormal flow:** missing, foreign, failed or wrong-type generations are rejected; a failed transaction leaves the old week unchanged.
  - **Rules:** applying a plan never writes the diary by itself.

d. Notifications

- **Function trigger:** Bell icon in the header (login required).
- **Function description:** Show messages such as report outcomes and post removals; mark as read or delete. Actor: Member.
- **Screen layout:** Non-screen function (runs in the back-end / background worker).
- **Function details:**
  - **Data:** notifications(content <= 500, is_read, created_at).
  - **Normal flow:** list newest first; mark one or all as read; delete own notifications.
  - **Abnormal flow:** acting on another user's notification returns not found.
  - **Rules:** notifications are created by the system only.

7. Administration

a. Dashboard and system health

- **Function trigger:** Login as ADMIN > /admin?tab=dashboard.

- **Function description:** Overview of platform activity and of the health of PostgreSQL and the AI service. Actor: Admin.
- **Screen layout:**

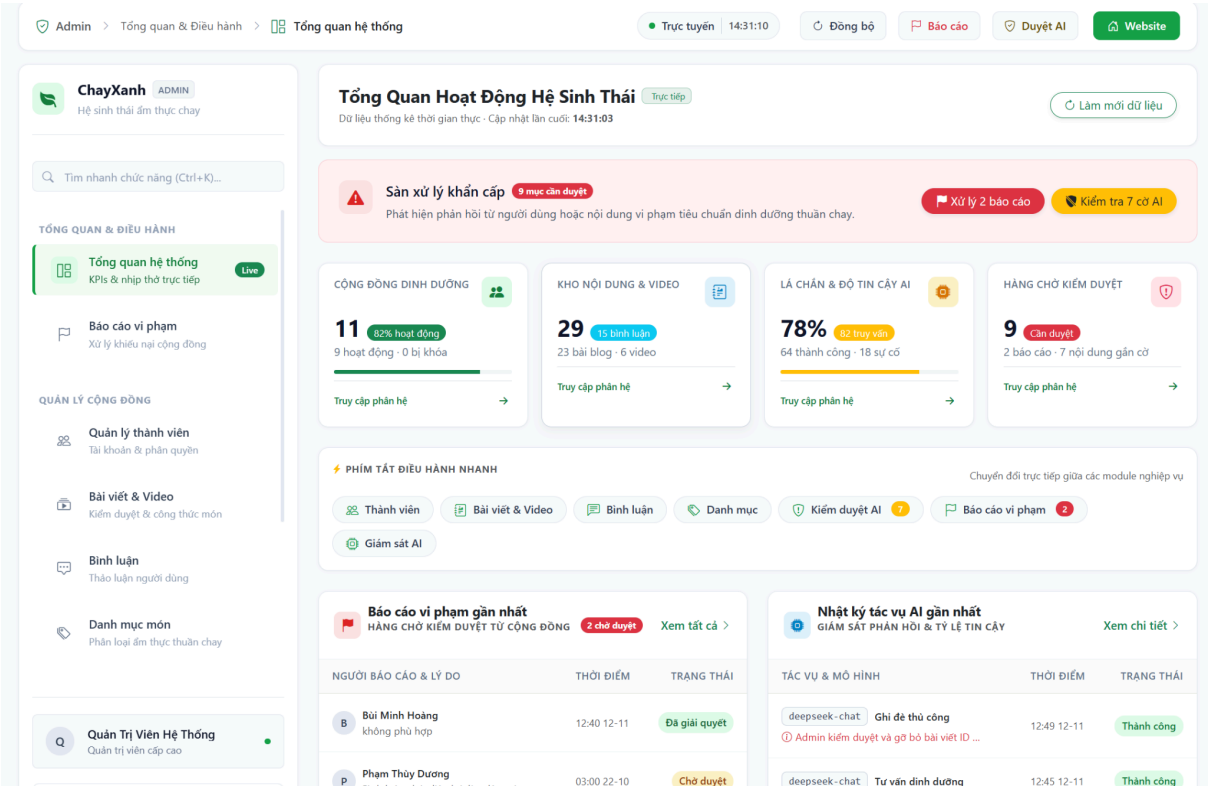


Figure 25. Admin dashboard

- **Function details:**
  - **Data:** counts of members, published posts, hidden-by-AI items, pending / resolved reports; health of database and AI /health endpoint.
  - **Normal flow:** statistics are computed from real records; urgent items link to their tabs.
  - **Abnormal flow:** an unreachable service is shown as DEGRADED / ERROR.
  - **Rules:** a healthy AI service process does not prove that the external LLM provider is available.

b. Member management

- **Function trigger:** /admin?tab=members.
- **Function description:** Search members, view their content, lock / unlock, delete and restore accounts. Actor: Admin.

- **Screen layout:**

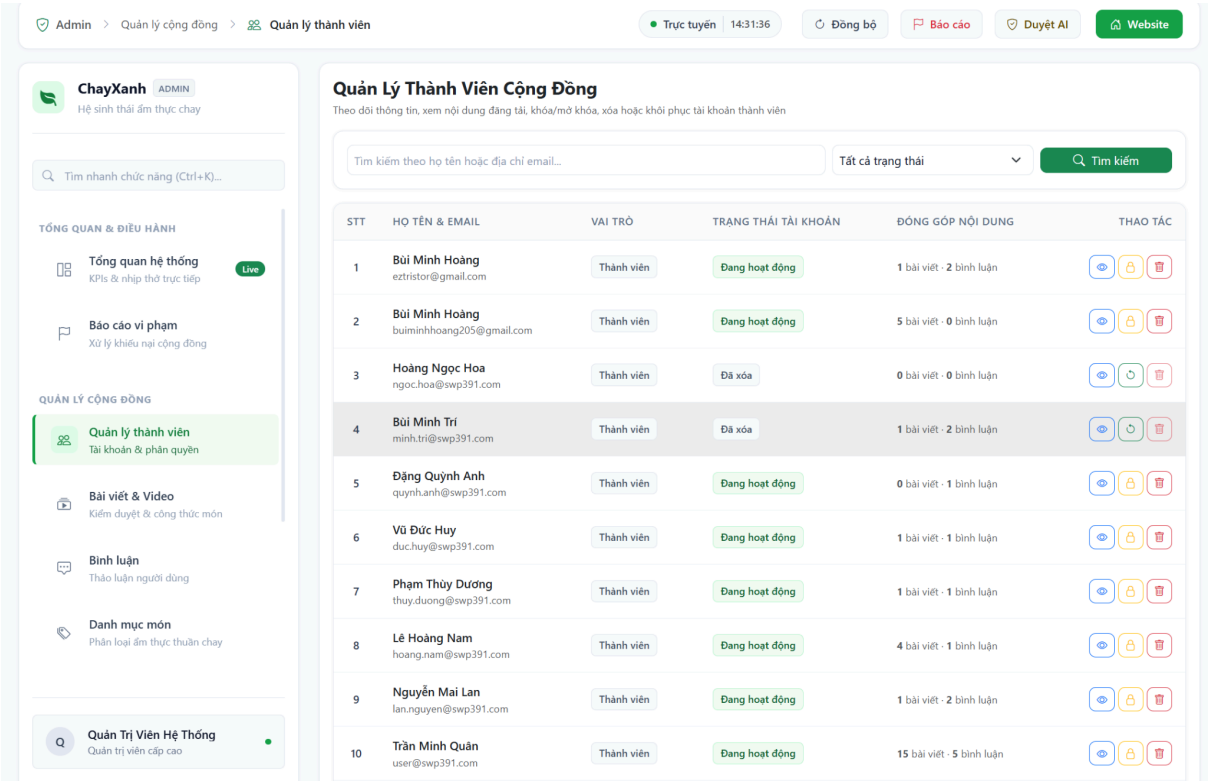


Figure 26. Admin - member management

- **Function details:**
  - **Data:** status ACTIVE / LOCKED / DELETED.
  - **Normal flow:** lock blocks sign-in; delete soft-deletes the account and its content in one transaction; restore reverses a deletion.
  - **Abnormal flow:** an administrator cannot delete his / her own account.
  - **Rules:** role editing is not offered.

c. Post and comment management

- **Function trigger:** /admin?tab=posts and /admin?tab=comments.
- **Function description:** Filter, edit, hide, restore or delete community content. Actor: Admin.

- **Screen layout:**

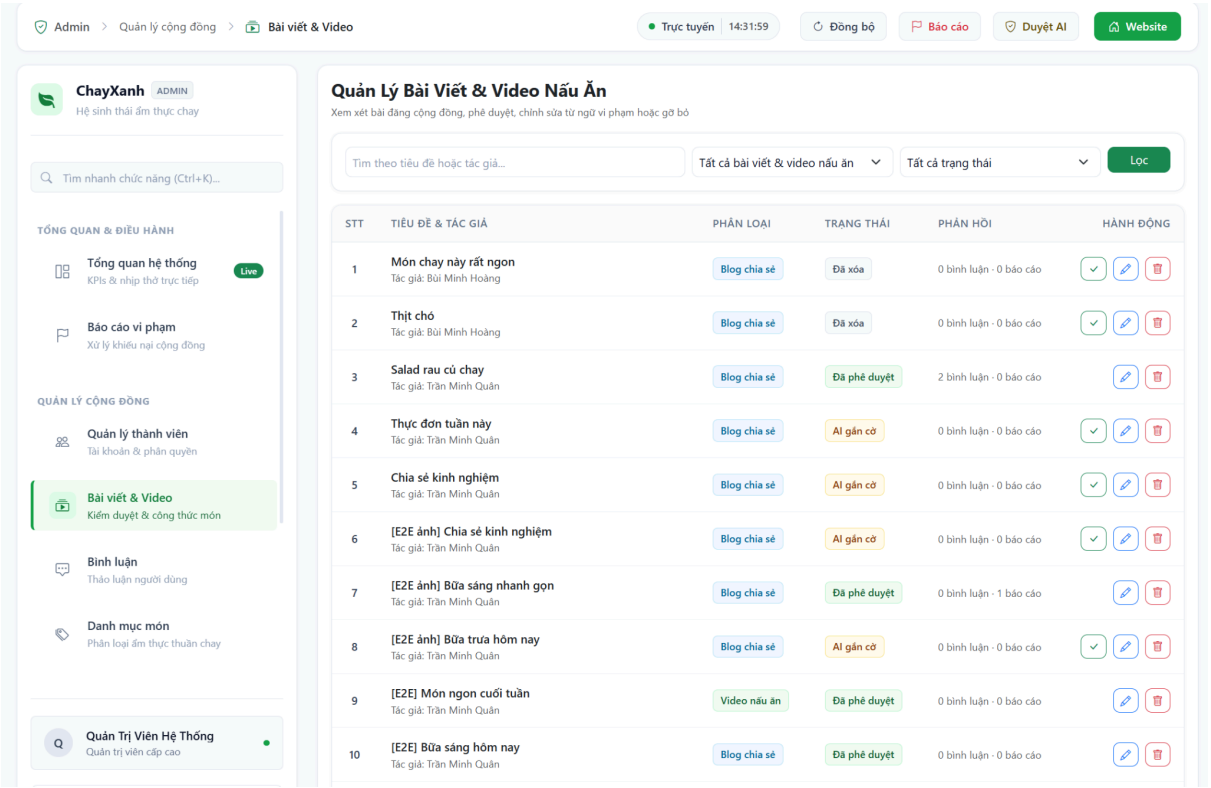


Figure 27. Admin - posts and videos management

- **Function details:**
  - **Data:** post type, status, keyword filters with pagination.
  - **Normal flow:** hide sets HIDDEN (restorable); delete sets DELETED and also deletes the post's comments.
  - **Abnormal flow:** acting on already deleted content returns a conflict message.
  - **Rules:** the comments tab offers the same actions for comments.

d. Category management

- **Function trigger:** /admin?tab=categories.
- **Function description:** Create, rename and delete categories used by recipes and posts. Actor: Admin.

- **Screen layout:**

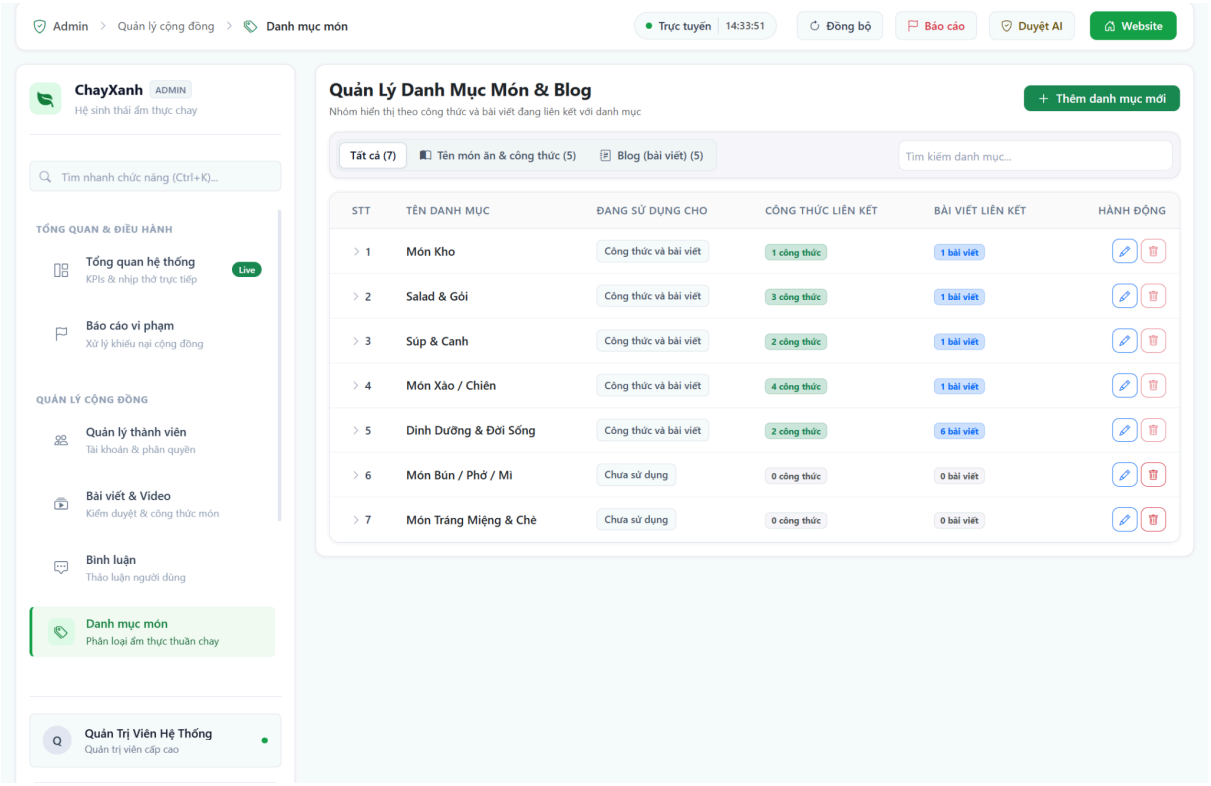


Figure 28. Admin - category management

- **Function details:**
  - **Data:** category name (unique, <= 100 characters).
  - **Normal flow:** changes are visible immediately in the recipe and community filters.
  - **Abnormal flow:** duplicate names return 409.
  - **Rules:** the table shows how many recipes and posts use each category.

e. AI moderation queue

- **Function trigger:** /admin?tab=ai-moderation.
- **Function description:** Review content that the AI flagged (HIDDEN_BY_AI), pending or failed moderation, then approve or remove it. Actor: Admin.

- **Screen layout:**

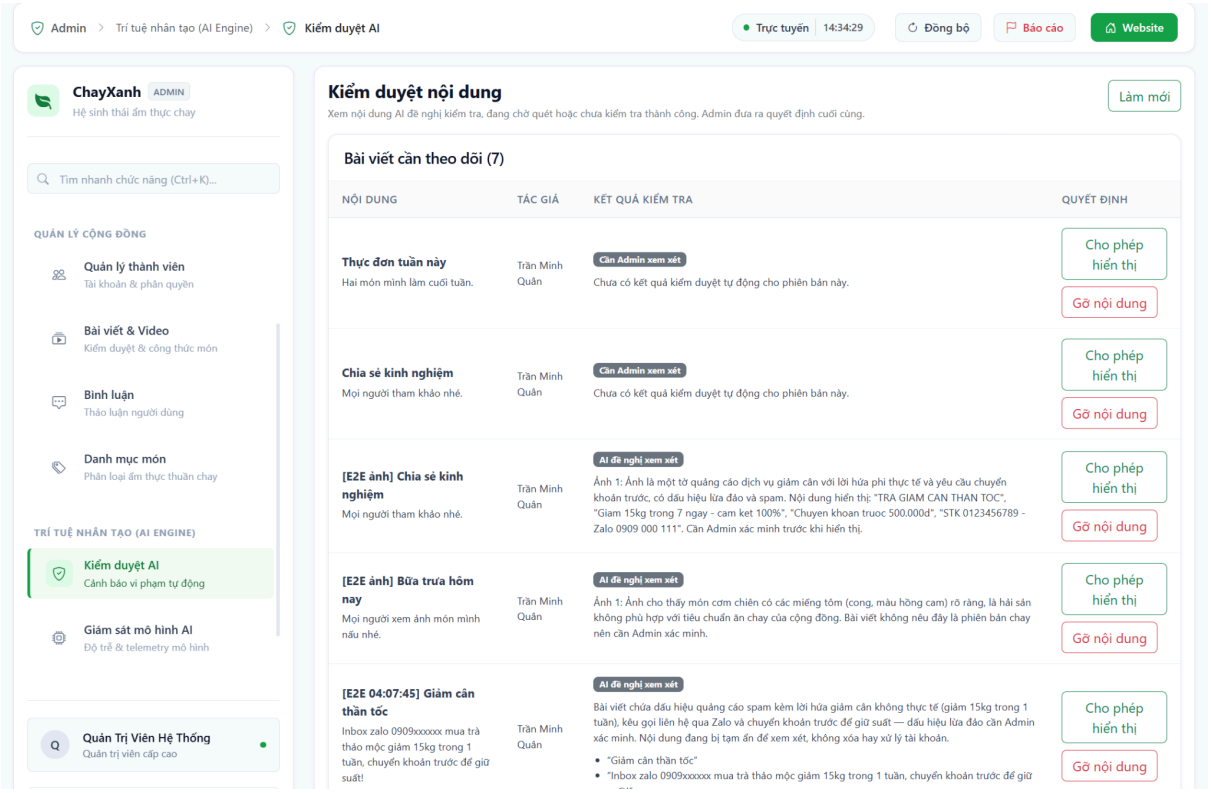


Figure 29. Admin - AI moderation queue

- **Function details:**
  - **Data:** flagged posts and comments with the AI findings.
  - **Normal flow:** approve publishes the post / activates the comment; remove sets DELETED.
  - **Abnormal flow:** content changed or removed meanwhile returns a conflict asking to refresh.
  - **Rules:** decisions are protected by the moderation version so stale results cannot override them.

f. Violation report handling

- **Function trigger:** /admin?tab=reports.
- **Function description:** Process member reports. Actor: Admin.

- **Screen layout:**

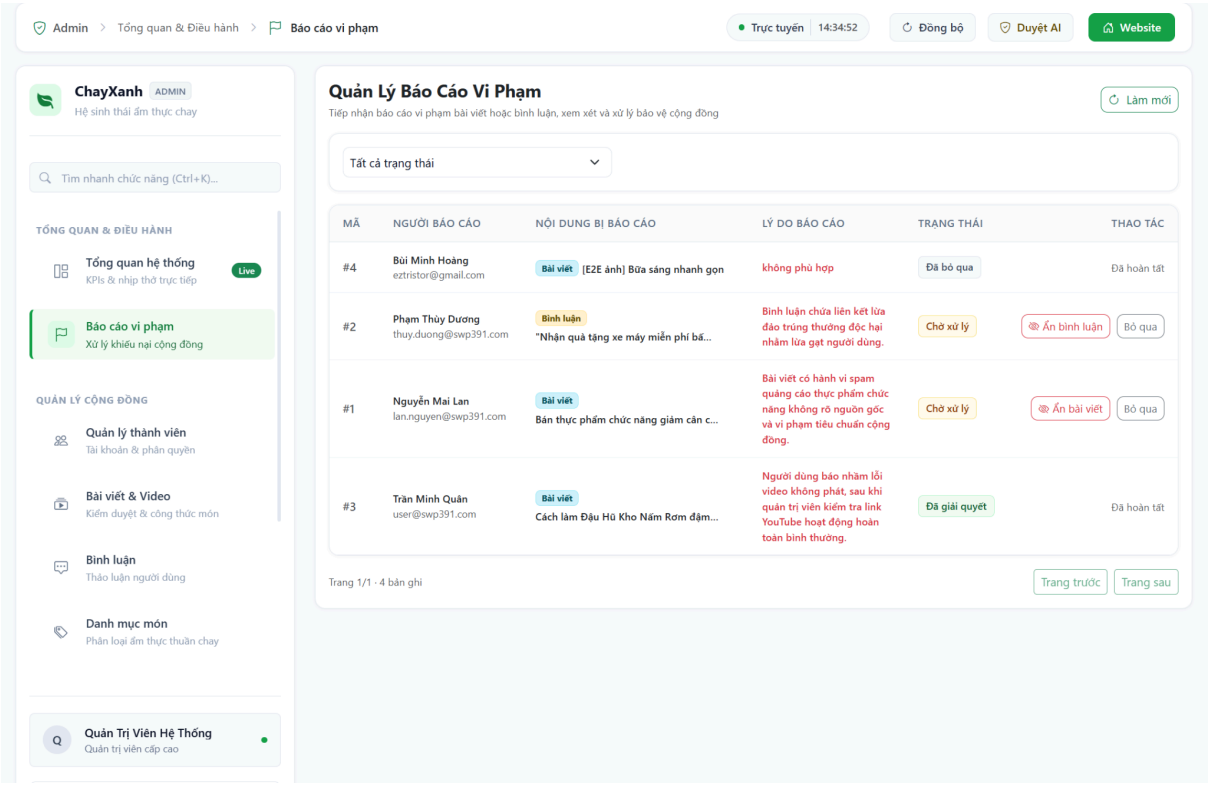


Figure 30. Admin - violation reports

- **Function details:**
  - **Data:** status filter PENDING / RESOLVED / DISMISSED; resolution action NONE or HIDE_CONTENT.
  - **Normal flow:** see Business Process 3.3.
  - **Abnormal flow:** unknown report id returns 404.
  - **Rules:** both reporter and (on removal) author receive notifications.

g. AI monitor and AI management

- **Function trigger:** /admin?tab=ai-monitor and /admin?tab=ai-management.
- **Function description:** Monitor AI models and logs, resolve failed logs manually, look up / reset guest quotas and clear the chatbot cache. Actor: Admin.

- **Screen layout:**

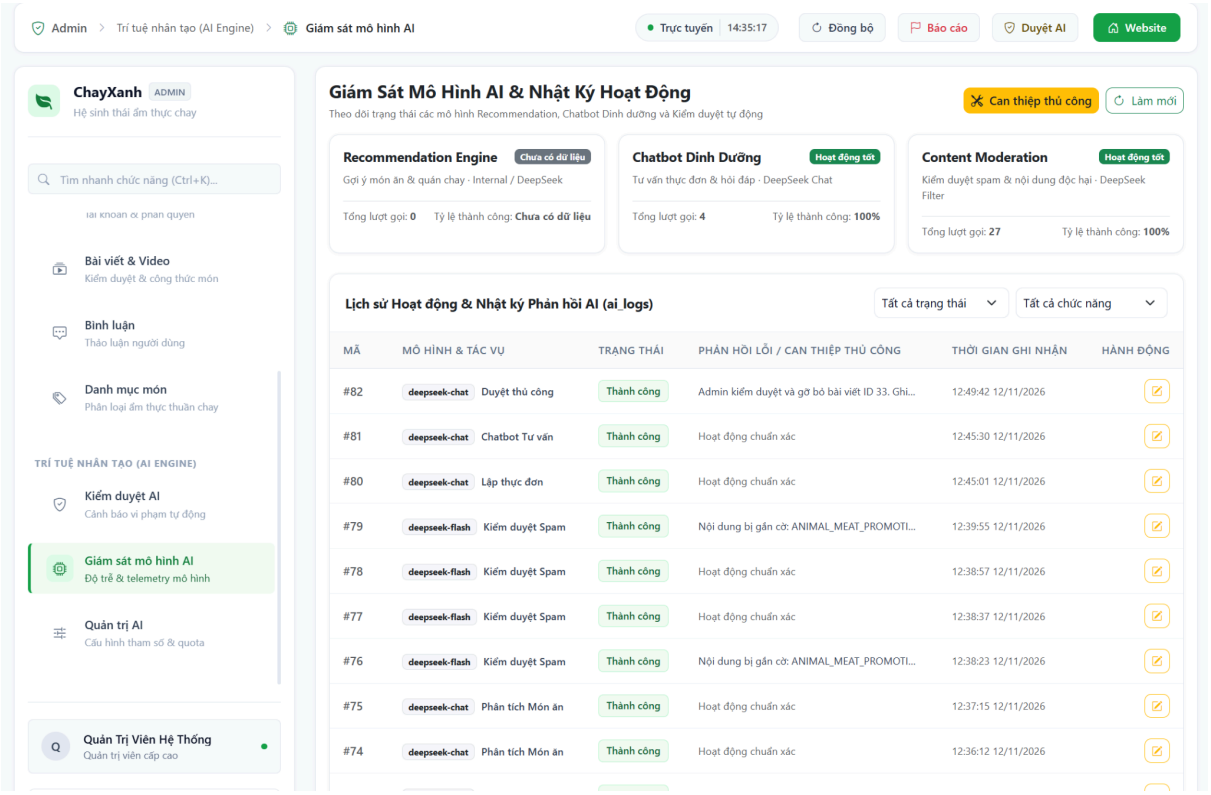


Figure 31. Admin - AI monitor

- **Function details:**
  - **Data:** ai_logs (model_name, action, status, error_feedback, created_at), ai statistics, guest id.
  - **Normal flow:** logs and statistics are read from real records; an intervention records RESOLVED_BY_ADMIN with the admin's feedback.
  - **Abnormal flow:** missing logs or an unreachable AI service are shown as errors.
  - **Rules:** no token-count or latency values are shown because they are not stored.

III. Database Design

1. Database Schema

The database is PostgreSQL, defined in server/prisma/schema.prisma and created with Prisma Migrate. Prisma models use PascalCase names; tables and columns are snake_case through @@map / @map. The 41 tables are grouped into six domains; each diagram shows primary keys (PK), foreign keys (FK) and the relationship cardinality.

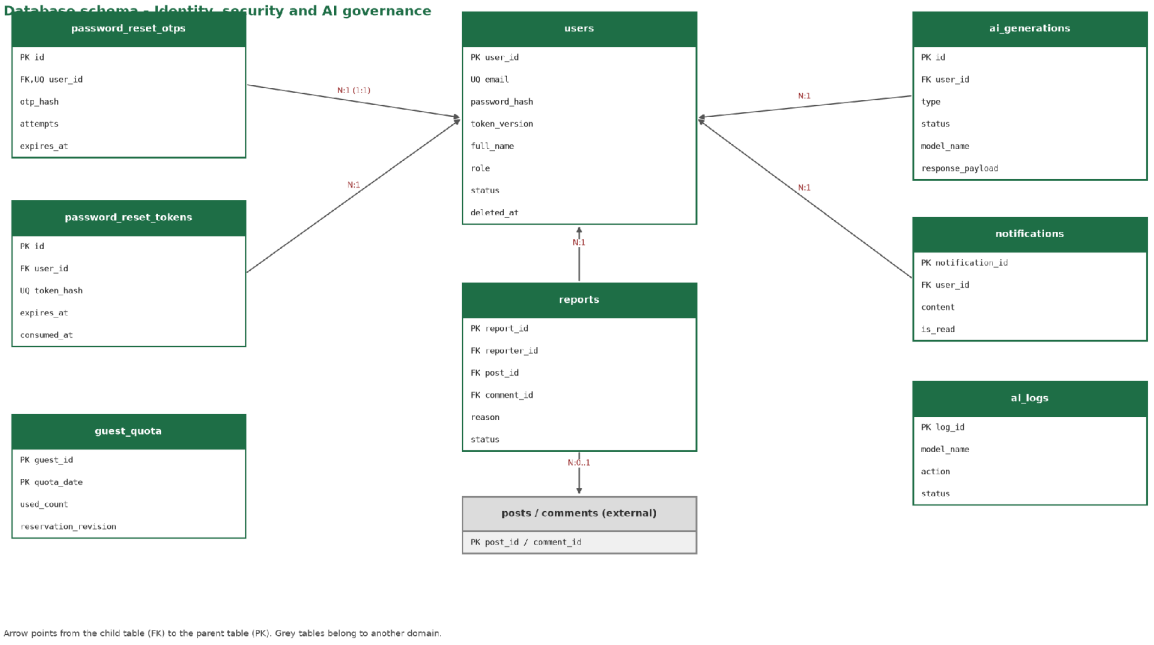


Figure 32. Database schema - Identity, security and AI governance

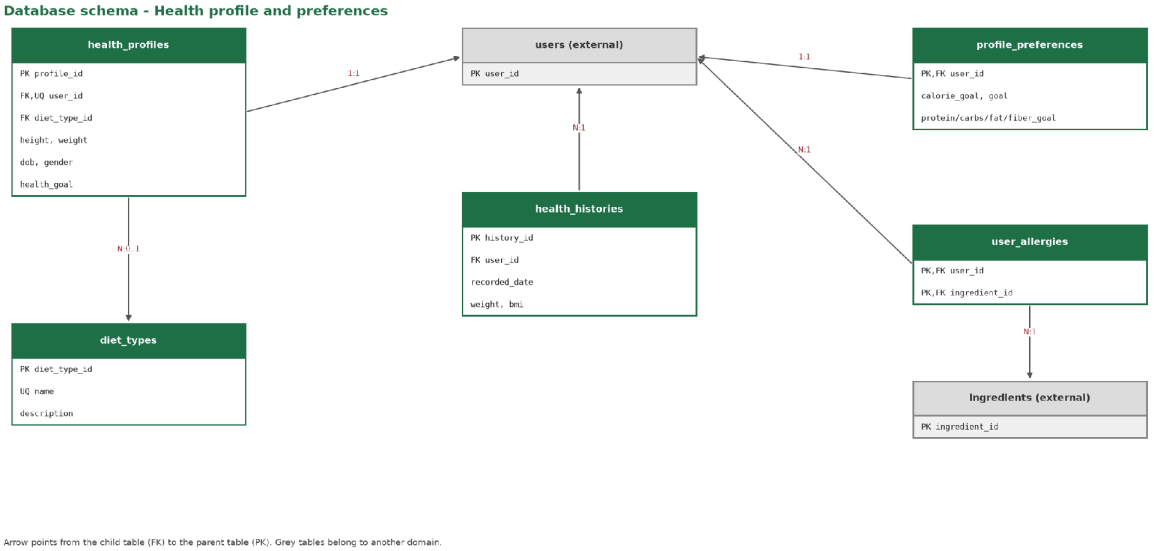


Figure 33. Database schema - Health profile and preferences

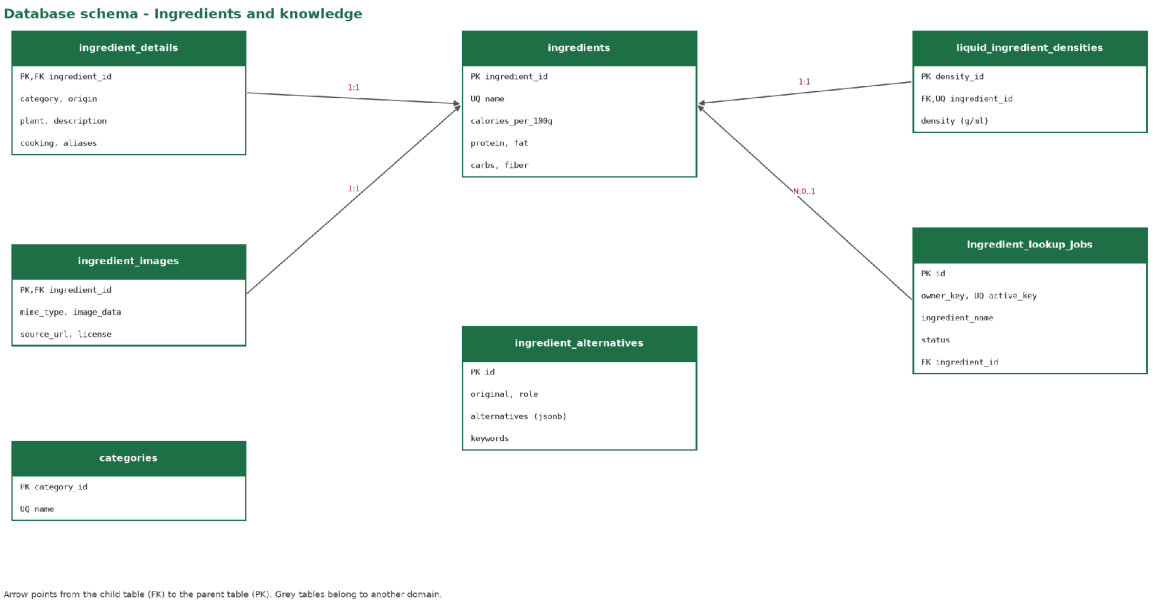


Figure 34. Database schema - Ingredients and knowledge

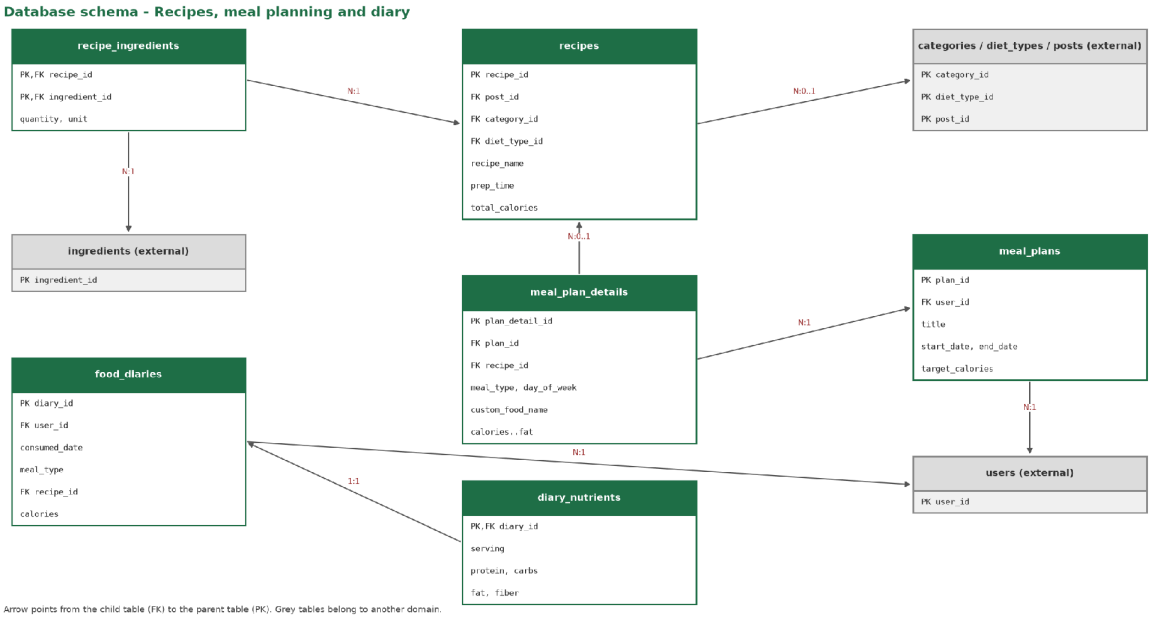


Figure 35. Database schema - Recipes, meal planning and diary

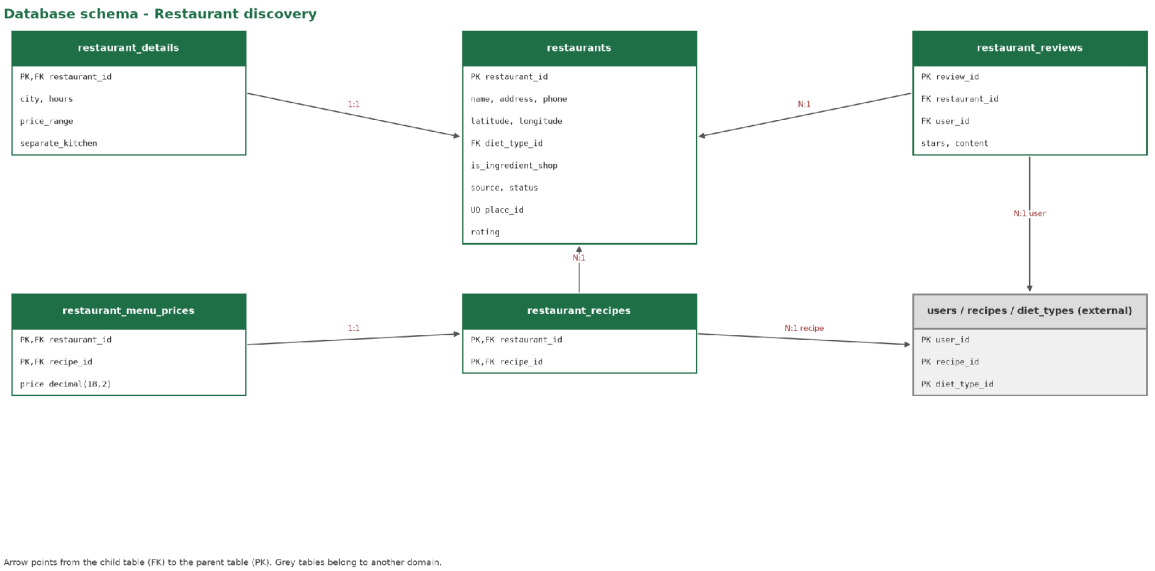


Figure 36. Database schema - Restaurant discovery

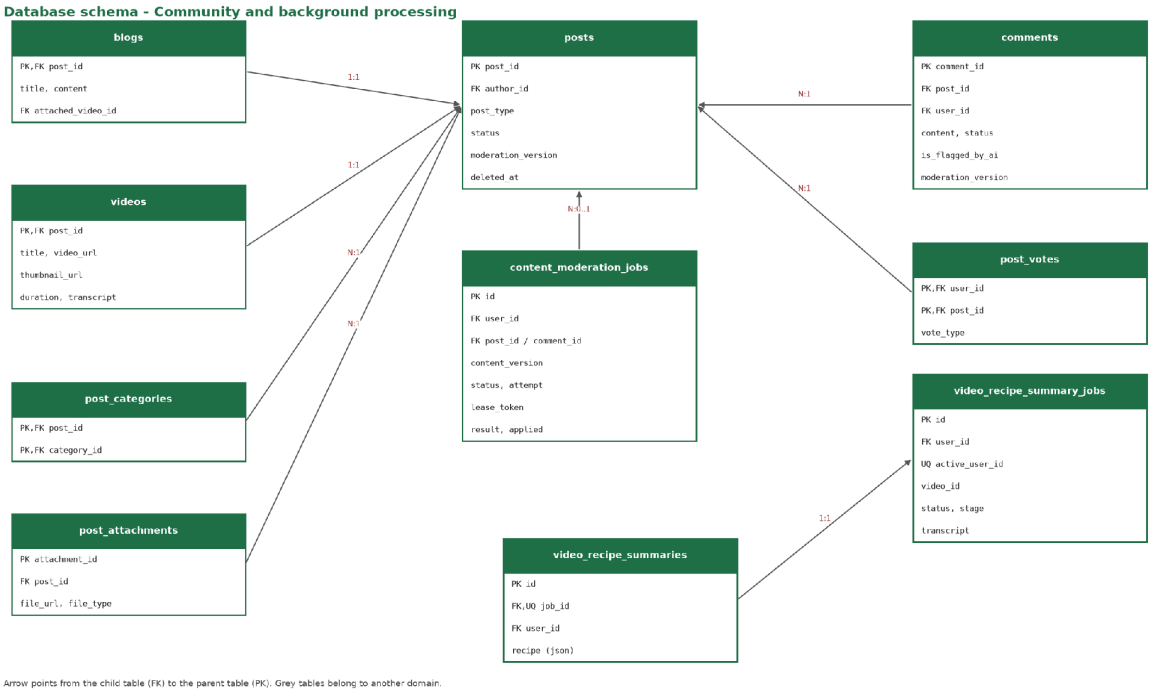


Figure 37. Database schema - Community and background processing

Table Description

No	Table	Description
1	users	Account credentials, role, token version, status and soft deletion.
2	password_reset_otps	One active hashed e-mail OTP per user with attempts and expiry.
3	password_reset_tokens	Hashed single-use password reset token and expiry.
4	guest_quota	Daily shared AI usage of a guest identity.
5	ai_generations	AI requests and responses of members (CHAT, MEAL_PLAN, FOOD_SCAN).
6	ai_logs	AI model / action / status logs for monitoring.
7	reports	Violation reports on posts or comments.
8	notifications	Notifications for users.

No	Table	Description
9	diet_types	Diet type catalogue used by profiles, recipes and restaurants.
10	health_profiles	Height, weight, birth date, gender, diet type and goal of one user.
11	health_histories	Weight and BMI history of a user.
12	user_allergies	Ingredients a user is allergic to (composite key).
13	profile_preferences	Calorie and macro / fiber goals of a user.
14	categories	Categories for recipes and posts.
15	ingredients	Ingredient name and nutrition per 100 g.
16	ingredient_details	Classification, origin, plant flag, description, cooking notes and aliases of an ingredient.
17	ingredient_images	Stored ingredient image with source and license attribution.
18	ingredient_alternatives	Vegetarian substitutions for a non-vegetarian ingredient.
19	liquid_ingredient_densities	Density (g/ml) of liquid ingredients for unit conversion.
20	ingredient_lookup_jobs	Background AI enrichment jobs for unknown ingredients.
21	recipes	Recipe name, description, image, preparation time, calories, instructions and links.
22	recipe_ingredients	Ingredient quantities of a recipe (composite key).
23	meal_plans	Weekly plan of a user (start / end date, target calories).
24	meal_plan_details	Meal slot of a plan: day, meal type, recipe or custom food and nutrients.
25	food_diaries	Food journal entry of a user by date and meal type.
26	diary_nutrients	Serving and nutrient snapshot of a diary entry.
27	restaurants	Vegetarian restaurant or ingredient shop with location, source and status.
28	restaurant_details	City, hours, price range and separate kitchen information.
29	restaurant_reviews	Member star rating and review text.
30	restaurant_recipes	Dishes served at a restaurant (composite key).
31	restaurant_menu_prices	Optional price of a restaurant dish.
32	posts	Community publication: author, type, status, moderation version, soft deletion.
33	blogs	Blog subtype of a post (title, content, optional attached video).
34	videos	Video subtype of a post (title, URL, thumbnail, duration, transcript).
35	post_categories	Categories of a post (composite key).
36	post_attachments	Uploaded files attached to a post.
37	comments	Flat comment on a post with status and AI flag.
38	post_votes	One vote per user per post.
39	content_moderation_jobs	Versioned AI moderation jobs with lease and result.
40	video_recipe_summary_jobs	YouTube recipe extraction jobs with status, stage and lease.
41	video_recipe_summaries	Structured recipe result of a completed video job.

2. Table Detail

Field name is the PostgreSQL column. Type and Size come from the Prisma native type (e.g. @db.VarChar(255)). Unique = Yes for a single-column primary key or a unique constraint. PK/FK marks key columns; Notes give the Prisma field name when it differs, the default value and the referenced table. Composite keys, composite unique constraints and indexes are listed under each table.

2.1 Identity, security and AI governance

users (model User)

Account credentials, role, token version, status and soft deletion.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	user_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	email	varchar	255	Yes	Yes		
3	password_hash	varchar	255		Yes		
4	token_version	integer	—		Yes		Prisma: tokenVersion; Default 0

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
5	full_name	varchar	255		Yes		
6	role	smallint	—		Yes		Default 2
7	status	varchar	20		Yes		Default "ACTIVE"
8	deleted_at	timestamptz(6)	—				
9	created_at	timestamptz(6)	—				Default now()

- Indexes: (email); (status, deleted_at).

password_reset_otps (model PasswordResetOtp)

One active hashed e-mail OTP per user with attempts and expiry.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	user_id	integer	—	Yes	Yes	FK	Prisma: userId; -> users.user_id (on delete Cascade)
3	otp_hash	varchar	64		Yes		Prisma: otpHash
4	token_version	integer	—		Yes		Prisma: tokenVersion
5	attempts	integer	—		Yes		Default 0
6	expires_at	timestamptz(6)	—		Yes		Prisma: expiresAt
7	last_sent_at	timestamptz(6)	—		Yes		Prisma: lastSentAt
8	consumed_at	timestamptz(6)	—				Prisma: consumedAt
9	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
10	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

password_reset_tokens (model PasswordResetToken)

Hashed single-use password reset token and expiry.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	user_id	integer	—		Yes	FK	Prisma: userId; -> users.user_id (on delete Cascade)
3	token_hash	varchar	64	Yes	Yes		Prisma: tokenHash
4	token_version	integer	—		Yes		Prisma: tokenVersion
5	expires_at	timestamptz(6)	—		Yes		Prisma: expiresAt
6	consumed_at	timestamptz(6)	—				Prisma: consumedAt
7	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
8	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

- Indexes: (user_id, expires_at).

guest_quota (model GuestQuota)

Daily shared AI usage of a guest identity.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	guest_id	varchar	100		Yes	PK	Prisma: guestId
2	quota_date	date	—		Yes	PK	Prisma: quotaDate

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
3	used_count	integer	—		Yes		Prisma: usedCount; Default 0
4	reservation_revision	integer	—		Yes		Prisma: reservationRevision; Default 0
5	updated_at	timestamp(6)	—		Yes		Prisma: updatedAt; Default now()

- Composite primary key: (guest_id, quota_date).

ai_generations (model AiGeneration)

AI requests and responses of members (CHAT, MEAL_PLAN, FOOD_SCAN).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	user_id	integer	—		Yes	FK	Prisma: userId; -> users.user_id (on delete Cascade)
3	type	enum AiGenerationType	—		Yes		
4	status	enum AiGenerationStatus	—		Yes		Default PROCESSING
5	model_name	varchar	100		Yes		Prisma: modelName
6	request_payload	jsonb	—		Yes		Prisma: requestPayload
7	prompt_messages	jsonb	—		Yes		Prisma: promptMessages
8	response_payload	jsonb	—				Prisma: responsePayload
9	error_code	varchar	100				Prisma: errorCode
10	error_message	text	—				Prisma: errorMessage
11	completed_at	timestamptz(6)	—				Prisma: completedAt
12	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
13	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

- Indexes: (user_id, created_at, id); (user_id, type, status, created_at, id).

ai_logs (model AiLog)

AI model / action / status logs for monitoring.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	log_id	integer	—	Yes	Yes	PK	Prisma: logId; Default autoincrement()
2	model_name	varchar	100		Yes		Prisma: modelName
3	action	varchar	100		Yes		
4	status	varchar	20		Yes		
5	error_feedback	text	—				Prisma: errorFeedback
6	created_at	timestamptz(6)	—				Prisma: createdAt; Default now()

reports (model Report)

Violation reports on posts or comments.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	report_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	reporter_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
3	post_id	integer	—			FK	-> posts.post_id (on delete Cascade)
4	comment_id	integer	—			FK	-> comments.comment_id (on delete Cascade)
5	reason	text	—		Yes		
6	status	varchar	30		Yes		Default "PENDING"
7	created_at	timestamptz(6)	—				Default now()

- Indexes: (reporter_id); (post_id); (comment_id); (status).

notifications (model Notification)

Notifications for users.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	notification_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	user_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
3	content	varchar	500		Yes		
4	is_read	boolean	—				Default false
5	created_at	timestamptz(6)	—				Default now()

2.2 Health profile and preferences

diet_types (model DietType)

Diet type catalogue used by profiles, recipes and restaurants.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	diet_type_id	smallint	—	Yes	Yes	PK	Default autoincrement()
2	name	varchar	100	Yes	Yes		
3	description	text	—				

health_profiles (model HealthProfile)

Height, weight, birth date, gender, diet type and goal of one user.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	profile_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	user_id	integer	—	Yes	Yes	FK	-> users.user_id (on delete Cascade)
3	diet_type_id	smallint	—			FK	-> diet_types.diet_type_id
4	height	real	—				
5	weight	real	—				
6	dob	date	—				
7	gender	varchar	20				
8	health_goal	varchar	255				

health_histories (model HealthHistory)

Weight and BMI history of a user.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	history_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	user_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
3	recorded_date	timestamptz(6)	—				Default now()
4	weight	real	—		Yes		
5	bmi	real	—				

user_allergies (model UserAllergy)

Ingredients a user is allergic to (composite key).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	user_id	integer	—		Yes	PK,FK	-> users.user_id (on delete Cascade)
2	ingredient_id	integer	—		Yes	PK,FK	-> ingredients.ingredient_id (on delete Cascade)

- Composite primary key: (user_id, ingredient_id).
- Indexes: (ingredient_id).

profile_preferences (model ProfilePreference)

Calorie and macro / fiber goals of a user.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	user_id	integer	—	Yes	Yes	PK,FK	-> users.user_id (on delete Cascade)
2	calorie_goal	real	—				
3	goal	text	—				
4	protein_goal	real	—				Default 70
5	carbs_goal	real	—				Default 230
6	fat_goal	real	—				Default 45
7	fiber_goal	real	—				Default 30

2.3 Ingredients and knowledge

categories (model Category)

Categories for recipes and posts.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	category_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	name	varchar	100	Yes	Yes		

ingredients (model Ingredient)

Ingredient name and nutrition per 100 g.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	ingredient_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	name	varchar	150	Yes	Yes		
3	calories_per_100g	real	—				Default 0
4	protein	real	—				Default 0
5	fat	real	—				Default 0

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
6	carbs	real	—				Default 0
7	fiber	real	—				Default 0

ingredient_details (model IngredientDetails)

Classification, origin, plant flag, description, cooking notes and aliases of an ingredient.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	ingredient_id	integer	—	Yes	Yes	PK,FK	-> ingredients.ingredient_id (on delete Cascade)
2	category	varchar	30		Yes		
3	origin	text	—		Yes		
4	plant	boolean	—				
5	description	text	—		Yes		
6	cooking	text	—		Yes		
7	aliases	text	—		Yes		Default ""
8	image_url	text	—				

ingredient_images (model IngredientImage)

Stored ingredient image with source and license attribution.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	ingredient_id	integer	—	Yes	Yes	PK,FK	-> ingredients.ingredient_id (on delete Cascade)
2	mime_type	varchar	50		Yes		
3	image_data	bytea	—		Yes		
4	attribution	text	—				
5	source_url	text	—				
6	license	text	—				
7	license_text	text	—				

ingredient_alternatives (model IngredientAlternative)

Vegetarian substitutions for a non-vegetarian ingredient.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	varchar	40	Yes	Yes	PK	
2	chip	text	—		Yes		
3	emoji	text	—		Yes		
4	original	text	—		Yes		
5	role	text	—		Yes		
6	alternatives	jsonb	—		Yes		
7	description	text	—		Yes		
8	keywords	text	—		Yes		

liquid_ingredient_densities (model LiquidIngredientDensity)

Density (g/ml) of liquid ingredients for unit conversion.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	density_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	ingredient_id	integer	—	Yes	Yes	FK	-> ingredients.ingredient_id (on delete Cascade)
3	density	real	—		Yes		

ingredient_lookup_jobs (model IngredientLookupJob)

Background AI enrichment jobs for unknown ingredients.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	owner_key	char	64		Yes		Prisma: ownerKey
3	active_key	char	64	Yes			Prisma: activeKey
4	ingredient_name	varchar	150		Yes		Prisma: ingredientName
5	status	enum IngredientLookupJobStatus	—		Yes		Default PENDING
6	ingredient_id	integer	—			FK	Prisma: ingredientId; -> ingredients.ingredient_id (on delete SetNull)
7	source_photo	bytea	—				Prisma: sourcePhoto
8	source_mime_type	varchar	50				Prisma: sourceMimeType
9	reservation	jsonb	—				
10	lease_token	uuid	—				Prisma: leaseToken
11	lease_expires_at	timestamptz(6)	—				Prisma: leaseExpiresAt
12	error_code	varchar	100				Prisma: errorCode
13	error_message	varchar	500				Prisma: errorMessage
14	completed_at	timestamptz(6)	—				Prisma: completedAt
15	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
16	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

- Indexes: (owner_key, created_at, id); (status, created_at, id); (status, lease_expires_at); (ingredient_id).

2.4 Recipes, meal planning and diary

recipes (model Recipe)

Recipe name, description, image, preparation time, calories, instructions and links.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	recipe_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	post_id	integer	—			FK	-> posts.post_id (on delete SetNull)
3	category_id	integer	—			FK	-> categories.category_id
4	diet_type_id	smallint	—			FK	-> diet_types.diet_type_id
5	recipe_name	varchar	255		Yes		
6	description	text	—				
7	image_url	varchar	500				

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
8	prep_time	integer	—				Default 0
9	total_calories	real	—				Default 0
10	instructions	text	—				

- Indexes: (post_id); (category_id); (diet_type_id).

recipe_ingredients (model RecipeIngredient)

Ingredient quantities of a recipe (composite key).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	recipe_id	integer	—		Yes	PK,FK	-> recipes.recipe_id (on delete Cascade)
2	ingredient_id	integer	—		Yes	PK,FK	-> ingredients.ingredient_id (on delete Cascade)
3	quantity	real	—		Yes		
4	unit	varchar	50		Yes		

- Composite primary key: (recipe_id, ingredient_id).
- Indexes: (ingredient_id).

meal_plans (model MealPlan)

Weekly plan of a user (start / end date, target calories).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	plan_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	user_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
3	title	varchar	255		Yes		Default "Thực đơn tuần"
4	start_date	date	—		Yes		
5	end_date	date	—		Yes		
6	target_calories	real	—				

- Indexes: (user_id, start_date).

meal_plan_details (model MealPlanDetail)

Meal slot of a plan: day, meal type, recipe or custom food and nutrients.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	plan_detail_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	plan_id	integer	—		Yes	FK	-> meal_plans.plan_id (on delete Cascade)
3	recipe_id	integer	—			FK	-> recipes.recipe_id (on delete Cascade)
4	meal_type	varchar	30		Yes		
5	day_of_week	smallint	—		Yes		
6	custom_food_name	varchar	255				
7	calories	real	—				
8	protein	real	—				
9	carbs	real	—				

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
10	fat	real	—				

- Indexes: (plan_id).

food_diaries (model FoodDiary)

Food journal entry of a user by date and meal type.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	diary_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	user_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
3	consumed_date	date	—				Default dbgenerated("CURRENT_DATE")
4	meal_type	varchar	30		Yes		
5	recipe_id	integer	—			FK	-> recipes.recipe_id
6	custom_food_name	varchar	255				
7	calories	real	—				Default 0

- Indexes: (user_id, consumed_date).

diary_nutrients (model DiaryNutrients)

Serving and nutrient snapshot of a diary entry.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	diary_id	integer	—	Yes	Yes	PK,FK	-> food_diaries.diary_id (on delete Cascade)
2	serving	text	—		Yes		Default "1 phần"
3	protein	real	—		Yes		Default 0
4	carbs	real	—		Yes		Default 0
5	fat	real	—		Yes		Default 0
6	fiber	real	—		Yes		Default 0

2.5 Restaurant discovery

restaurants (model Restaurant)

Vegetarian restaurant or ingredient shop with location, source and status.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	restaurant_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	name	varchar	255		Yes		
3	address	varchar	255		Yes		
4	phone	varchar	50				
5	image_url	varchar	500				
6	rating	real	—				Default 4.5
7	latitude	decimal	10, 7				
8	longitude	decimal	10, 7				
9	diet_type_id	smallint	—			FK	-> diet_types.diet_type_id

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
10	is_ingredient_shop	boolean	—		Yes		Prisma: isIngredientShop; Default false
11	source	varchar	20		Yes		Default "local"
12	status	varchar	20		Yes		Default "APPROVED"
13	place_id	varchar	100	Yes			
14	created_at	timestamptz(6)	—				Default now()
15	updated_at	timestamptz(6)	—				Auto-updated (@updatedAt)

- Indexes: (latitude, longitude); (diet_type_id); (status, latitude, longitude); (diet_type_id, latitude, longitude).

restaurant_details (model RestaurantDetails)

City, hours, price range and separate kitchen information.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	restaurant_id	integer	—	Yes	Yes	PK,FK	-> restaurants.restaurant_id (on delete Cascade)
2	city	text	—		Yes		
3	hours	text	—				
4	price_range	text	—				
5	separate_kitchen	boolean	—				

restaurant_reviews (model RestaurantReview)

Member star rating and review text.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	review_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	restaurant_id	integer	—		Yes	FK	-> restaurants.restaurant_id (on delete Cascade)
3	user_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
4	stars	smallint	—		Yes		
5	content	text	—		Yes		
6	created_at	timestamptz(6)	—		Yes		Default now()

- Indexes: (restaurant_id); (user_id).

restaurant_recipes (model RestaurantRecipe)

Dishes served at a restaurant (composite key).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	restaurant_id	integer	—		Yes	PK,FK	-> restaurants.restaurant_id (on delete Cascade)
2	recipe_id	integer	—		Yes	PK,FK	-> recipes.recipe_id (on delete Cascade)

- Composite primary key: (restaurant_id, recipe_id).

restaurant_menu_prices (model RestaurantMenuPrice)

Optional price of a restaurant dish.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	restaurant_id	integer	—		Yes	PK,FK	-> restaurant_recipes.restaurant_id (on delete Cascade)
2	recipe_id	integer	—		Yes	PK,FK	-> restaurant_recipes.recipe_id (on delete Cascade)
3	price	decimal	18, 2				

- Composite primary key: (restaurant_id, recipe_id).

2.6 Community and background processing

posts (model Post)

Community publication: author, type, status, moderation version, soft deletion.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	post_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	author_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
3	post_type	varchar	20		Yes		
4	status	varchar	30		Yes		Default "PUBLISHED"
5	status_before_account_deletion	varchar	30				Prisma: statusBeforeAccountDeletion
6	moderation_version	integer	—		Yes		Prisma: moderationVersion; Default 0
7	deleted_at	timestamptz(6)	—				
8	created_at	timestamptz(6)	—				Default now()

- Indexes: (author_id); (status); (status, deleted_at).

blogs (model Blog)

Blog subtype of a post (title, content, optional attached video).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	post_id	integer	—	Yes	Yes	PK,FK	-> posts.post_id (on delete Cascade)
2	title	varchar	255		Yes		
3	content	text	—		Yes		
4	attached_video_id	integer	—			FK	-> videos.post_id (on delete SetNull)

videos (model Video)

Video subtype of a post (title, URL, thumbnail, duration, transcript).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	post_id	integer	—	Yes	Yes	PK,FK	-> posts.post_id (on delete Cascade)
2	title	varchar	255		Yes		
3	video_url	varchar	500		Yes		
4	thumbnail_url	varchar	500				
5	duration	integer	—				Default 0
6	transcript	text	—				

post_categories (model PostCategory)

Categories of a post (composite key).

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	post_id	integer	—		Yes	PK,FK	-> posts.post_id (on delete Cascade)
2	category_id	integer	—		Yes	PK,FK	-> categories.category_id (on delete Cascade)

- Composite primary key: (post_id, category_id).
- Indexes: (category_id).

post_attachments (model PostAttachment)

Uploaded files attached to a post.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	attachment_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	post_id	integer	—		Yes	FK	-> posts.post_id (on delete Cascade)
3	file_url	varchar	500		Yes		
4	file_type	varchar	20		Yes		
5	file_name	varchar	255				
6	created_at	timestamptz(6)	—				Default now()

- Indexes: (post_id).

comments (model Comment)

Flat comment on a post with status and AI flag.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	comment_id	integer	—	Yes	Yes	PK	Default autoincrement()
2	post_id	integer	—		Yes	FK	-> posts.post_id (on delete Cascade)
3	user_id	integer	—		Yes	FK	-> users.user_id (on delete Cascade)
4	content	text	—		Yes		
5	is_flagged_by_ai	boolean	—				Default false
6	status	varchar	30		Yes		Default "ACTIVE"
7	status_before_account_deletion	varchar	30				Prisma: statusBeforeAccountDeletion
8	moderation_version	integer	—		Yes		Prisma: moderationVersion; Default 0
9	deleted_at	timestamptz(6)	—				
10	created_at	timestamptz(6)	—				Default now()

- Indexes: (post_id); (user_id); (status, deleted_at).

post_votes (model PostVote)

One vote per user per post.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	user_id	integer	—		Yes	PK,FK	-> users.user_id (on delete Cascade)
2	post_id	integer	—		Yes	PK,FK	-> posts.post_id (on delete Cascade)
3	vote_type	varchar	20		Yes		

- Composite primary key: (user_id, post_id).

content_moderation_jobs (model ContentModerationJob)

Versioned AI moderation jobs with lease and result.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	user_id	integer	—		Yes	FK	Prisma: userId; -> users.user_id (on delete Cascade)
3	post_id	integer	—			FK	Prisma: postId; -> posts.post_id (on delete Cascade)
4	comment_id	integer	—			FK	Prisma: commentId; -> comments.comment_id (on delete Cascade)
5	content_version	integer	—		Yes		Prisma: contentVersion
6	title	varchar	255				
7	content	text	—		Yes		
8	image_urls	text[]	—		Yes		Prisma: imageUrls; Default []
9	status	enum ContentModerationJobStatus	—		Yes		Default PENDING
10	attempt	integer	—		Yes		Default 0
11	available_at	timestamptz(6)	—		Yes		Prisma: availableAt; Default now()
12	lease_token	uuid	—				Prisma: leaseToken
13	lease_expires_at	timestamptz(6)	—				Prisma: leaseExpiresAt
14	result	jsonb	—				
15	applied	boolean	—		Yes		Default false
16	error_code	varchar	100				Prisma: errorCode
17	error_message	varchar	500				Prisma: errorMessage
18	completed_at	timestamptz(6)	—				Prisma: completedAt
19	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
20	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

- Composite unique: (post_id, content_version).
- Composite unique: (comment_id, content_version).
- Indexes: (user_id); (post_id); (comment_id); (status, available_at, created_at, id); (status, lease_expires_at).

video_recipe_summary_jobs (model VideoRecipeSummaryJob)

YouTube recipe extraction jobs with status, stage and lease.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	user_id	integer	—		Yes	FK	Prisma: userId; -> users.user_id (on delete Cascade)
3	active_user_id	integer	—	Yes			Prisma: activeUserId
4	video_id	varchar	11		Yes		Prisma: videoId
5	video_url	varchar	255		Yes		Prisma: videoUrl
6	pipeline_version	varchar	20		Yes		Prisma: pipelineVersion
7	status	enum VideoRecipeSummaryJobStatus	—		Yes		Default PENDING
8	stage	enum VideoRecipeSummaryJobStage	—		Yes		Default VALIDATING_VIDEO
9	attempt	integer	—		Yes		Default 0
10	available_at	timestamptz(6)	—		Yes		Prisma: availableAt; Default now()
11	lease_token	uuid	—				Prisma: leaseToken
12	lease_expires_at	timestamptz(6)	—				Prisma: leaseExpiresAt
13	metadata	jsonb	—				
14	transcript	jsonb	—				
15	error_code	varchar	100				Prisma: errorCode
16	error_message	varchar	500				Prisma: errorMessage
17	completed_at	timestamptz(6)	—				Prisma: completedAt
18	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
19	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

- Composite unique: (user_id, video_id, pipeline_version).
- Indexes: (status, available_at, created_at, id); (status, lease_expires_at); (user_id, created_at, id).

video_recipe_summaries (model VideoRecipeSummary)

Structured recipe result of a completed video job.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	id	uuid	—	Yes	Yes	PK	Default uuid()
2	job_id	uuid	—	Yes	Yes	FK	Prisma: jobId; -> video_recipe_summary_jobs.id (on delete Cascade)
3	user_id	integer	—		Yes	FK	Prisma: userId; -> users.user_id (on delete Cascade)
4	video_id	varchar	11		Yes		Prisma: videoId
5	video_url	varchar	255		Yes		Prisma: videoUrl
6	recipe	jsonb	—		Yes		
7	created_at	timestamptz(6)	—		Yes		Prisma: createdAt; Default now()
8	updated_at	timestamptz(6)	—		Yes		Prisma: updatedAt; Auto-updated (@updatedAt)

- Indexes: (user_id, created_at, id).

2.7 Enumerations

Enum	Values	Used by
AiGenerationType	CHAT, MEAL_PLAN, FOOD_SCAN	ai_generations.type
AiGenerationStatus	PROCESSING, COMPLETED, FAILED	ai_generations.status
ContentModerationJobStatus	PENDING, PROCESSING, COMPLETED, FAILED	content_moderation_jobs.status
IngredientLookupJobStatus	PENDING, PROCESSING, COMPLETED, FAILED	ingredient_lookup_jobs.status
VideoRecipeSummaryJobStatus	PENDING, PROCESSING, COMPLETED, FAILED	video_recipe_summary_jobs.status
VideoRecipeSummaryJobStage	VALIDATING_VIDEO, FETCHING_CAPTIONS, FETCHING_AUDIO, TRANSCRIBING, SUMMARIZING, SAVING_RESULT	video_recipe_summary_jobs.stage

String status columns (users.status, posts.status, comments.status, reports.status) take the values defined in server/src/constants/status.js and described in section I.6.